

乐客来福商户 V1.0 源程序鉴别材料

著作权人: 来富网络(宁晋)有限公司

开发完成日期: 2026年4月14日

软件语言: Dart

=====

【前段源程序(第 1 页 ~ 第 30 页, 第 1 行 ~ 第 1500 行)】

=====

第 01 页 第 1~50 行

=====

```
1
2 // =====
3 // 文件: lib/app.dart
4 // =====
5 import 'package:flutter/material.dart';
6 import 'package:flutter_riverpod/flutter_riverpod.dart';
7 import 'package:go_router/go_router.dart';
8 import 'package:merchant_app/config/theme.dart';
9 import 'package:merchant_app/core/service/navigation_service.dart';
10 import 'package:merchant_app/features/auth/auth_provider.dart';
11 import 'package:merchant_app/features/auth/bind_code_page.dart';
12 import 'package:merchant_app/features/auth/landing_page.dart';
13 import 'package:merchant_app/features/order/order_list_page.dart';
14 import 'package:merchant_app/features/settings/settings_page.dart';
15 import 'package:merchant_app/features/settings/permission_guide_page.dart';
16 import 'package:merchant_app/features/settings/about_page.dart';
17 import 'package:merchant_app/features/settings/agreement_page.dart';
18 import 'package:merchant_app/features/printer/bluetooth_printer_page.dart';
19 import 'package:merchant_app/features/order/order_detail_page.dart';
20 import 'package:merchant_app/models/order.dart';
21
22 final routerProvider = Provider<GoRouter>((ref) {
23   final routerNotifier = ref.watch(routerNotifierProvider);
24
25   return GoRouter(
26     navigatorKey: rootNavigatorKey,
27     initialLocation: '/',
28     refreshListenable: routerNotifier,
29     redirect: routerNotifier.redirect,
30     routes: [
31       GoRoute(
32         path: '/',
33         builder: (context, state) => const OrderListPage(),
34         routes: [
35           GoRoute(
36             path: 'order-detail',
37             builder: (context, state) {
```

```
38         final order = state.extra as OrderModel;
39         return OrderDetailPage(order: order);
40     },
41 ),
42 ],
43 ),
44 GoRoute(
45     path: '/welcome',
46     builder: (context, state) => const LandingPage(),
47 ),
48 GoRoute(
49     path: '/login',
50     builder: (context, state) => const BindCodePage(),
```

第 02 页 第 51~100 行

```
51     ),
52
53     GoRoute(
54         path: '/settings',
55         builder: (context, state) => const SettingsPage(),
56         routes: [
57             GoRoute(
58                 path: 'permission-guide',
59                 builder: (context, state) => const PermissionGuidePage(),
60             ),
61             GoRoute(
62                 path: 'about',
63                 builder: (context, state) => const AboutPage(),
64             ),
65             GoRoute(
66                 path: 'agreement/:type',
67                 builder: (context, state) => AgreementPage(
68                     agreementType: state.pathParameters['type'] ?? '',
69                 ),
70             ),
71         ],
72     ),
73     GoRoute(
74         path: '/printer',
75         builder: (context, state) => const BluetoothPrinterPage(),
76     ),
77 ],
78 );
79 });
80
81 final routerNotifierProvider = Provider<RouterNotifier>((ref) {
82     return RouterNotifier(ref);
```

```
83 });
84
85 class RouterNotifier extends ChangeNotifier {
86   final Ref _ref;
87
88   RouterNotifier(this._ref) {
89     _ref.listen(authProvider, (previous, next) {
90       if (previous?.isAuthenticated != next.isAuthenticated || previous?.isLoading != next.isLoading) {
91         notifyListeners();
92       }
93     });
94   }
95
96   String? redirect(BuildContext context, GoRouterState state) {
97     final authState = _ref.read(authProvider);
98     final currentPath = state.matchedLocation;
99     final isLoggingIn = currentPath == '/login';
100    final isWelcome = currentPath == '/welcome';
```

=====
第 03 页 第 101~150 行
=====

```
101    final isSettings = currentPath.startsWith('/settings');
102    final isPrinter = currentPath == '/printer';
103    final isOrderDetail = currentPath == '/order-detail';
104
105    debugPrint('Router: Redirect check - Auth: ${authState.isAuthenticated}, Loading: $
{authState.isLoading}, Current: $currentPath');
106
107    // If we are checking initial auth, don't redirect yet
108    if (authState.isLoading && authState.accessToken == null) {
109      return null;
110    }
111
112    if (!authState.isAuthenticated) {
113      if (isOrderDetail) {
114        return '/';
115      }
116
117      if (currentPath == '/' || isWelcome || isLoggingIn || isSettings || isPrinter) {
118        return null;
119      }
120
121      return '/';
122    }
123
124    // If authenticated but on auth pages, go to home
125    if (isWelcome || isLoggingIn) {
126      return '/';
```

```
127     }
128
129
130     return null;
131   }
132 }
133
134 class MerchantApp extends ConsumerWidget {
135   const MerchantApp({super.key});
136
137   @override
138   Widget build(BuildContext context, WidgetRef ref) {
139     final router = ref.watch(routerProvider);
140     final authState = ref.watch(authProvider);
141     final appTitle = (authState.merchantName != null && authState.merchantName!.trim().isEmpty)
142       ? authState.merchantName!.trim()
143       : '商户工作台';
144
145     return MaterialApp.router(
146       title: appTitle,
147       theme: AppTheme.lightTheme,
148       routerConfig: router,
149       debugShowCheckedModeBanner: false,
150       builder: (context, child) {
```

=====

第 04 页 第 151~200 行

=====

```
151     // Show a global loading overlay instead of replacing the entire app
152     return Stack(
153       children: [
154         child ?? const SizedBox.shrink(),
155         if (authState.isLoading)
156           Container(
157             color: Colors.black26,
158             child: const Center(child: CircularProgressIndicator()),
159           ),
160       ],
161     );
162   },
163 );
164 }
165 }
166
167
168
169 // =====
170 // 文件: lib/config/env.dart
171 // =====
```

```
172 class Env {
173   static const String apiBaseUrl = String.fromEnvironment(
174     'API_BASE_URL',
175     defaultValue: 'https://llapi.merrydance.cn/v1',
176   );
177
178   static const String wsUrl = String.fromEnvironment(
179     'WS_URL',
180     defaultValue: 'wss://llapi.merrydance.cn/v1/ws',
181   );
182
183   static const String jpushAppKey = String.fromEnvironment(
184     'JPUSH_APP_KEY',
185     defaultValue: '54cf8f6007b4b3338548d006',
186   );
187
188   static const String jpushChannel = 'developer-default';
189
190   static const bool isDebug = !bool.fromEnvironment('dart.vm.product');
191 }
192
193
194 // =====
195 // 文件: lib/config/theme.dart
196 // =====
197 import 'package:flutter/material.dart';
198
199 class AppColors {
200   static const primary = Color(0xFF006B31);
```

=====

第 05 页 第 201~250 行

=====

```
201   static const primaryContainer = Color(0xFF038740);
202   static const secondary = Color(0xFFFC820C);
203   static const tertiary = Color(0xFFB6171E);
204   static const surface = Color(0xFFFF5FBF1);
205   static const surfaceLow = Color(0xFFEFF5EC);
206   static const surfaceLowest = Color(0xFFFFFFFF);
207   static const onSurface = Color(0xFF171D17);
208   static const onSurfaceVariant = Color(0xFF3E4A3F);
209   static const outlineVariant = Color(0xFFBDCABB);
210   static const warningSoft = Color(0xFFFFF1E3);
211   static const dangerSoft = Color(0xFFDE9E8);
212 }
213
214 class AppSpacing {
215   static const xs = 4.0;
216   static const sm = 8.0;
```

```
217     static const md = 12.0;
218     static const lg = 16.0;
219     static const xl = 24.0;
220     static const xxl = 32.0;
221 }
222
223 class AppRadius {
224     static const md = 16.0;
225     static const lg = 20.0;
226     static const xl = 24.0;
227     static const pill = 999.0;
228 }
229
230 class AppTheme {
231     static const primaryColor = AppColors.primary;
232     static const accentColor = AppColors.secondary;
233     static const backgroundColor = AppColors.surface;
234     static const cardColor = AppColors.surfaceLowest;
235     static const textPrimary = AppColors.onSurface;
236     static const textSecondary = AppColors.onSurfaceVariant;
237
238     static ThemeData get lightTheme {
239         final base = ThemeData(useMaterial3: true);
240
241         return ThemeData(
242             useMaterial3: true,
243             colorScheme: ColorScheme.fromSeed(
244                 seedColor: primaryColor,
245                 primary: primaryColor,
246                 secondary: accentColor,
247                 tertiary: AppColors.tertiary,
248                 surface: backgroundColor,
249                 surfaceContainerLow: AppColors.surfaceLow,
250                 surfaceContainerLowest: AppColors.surfaceLowest,
```

=====

第 06 页 第 251~300 行

=====

```
251         onSurface: textPrimary,
252         onSurfaceVariant: textSecondary,
253         outlineVariant: AppColors.outlineVariant,
254     ),
255     scaffoldBackgroundColor: backgroundColor,
256     textTheme: base.textTheme.copyWith(
257         headlineMedium: base.textTheme.headlineMedium?.copyWith(
258             fontSize: 28,
259             fontWeight: FontWeight.w700,
260             color: textPrimary,
261         ),
```

```
262         titleLarge: base.textTheme.titleLarge?.copyWith(
263             fontSize: 22,
264             fontWeight: FontWeight.w700,
265             color: textPrimary,
266         ),
267         bodyLarge: base.textTheme.bodyLarge?.copyWith(
268             fontSize: 16,
269             height: 1.45,
270             color: textPrimary,
271         ),
272         bodyMedium: base.textTheme.bodyMedium?.copyWith(
273             fontSize: 14,
274             height: 1.4,
275             color: textPrimary,
276         ),
277         labelMedium: base.textTheme.labelMedium?.copyWith(
278             fontSize: 12,
279             fontWeight: FontWeight.w600,
280             color: textSecondary,
281         ),
282     ),
283     appBarTheme: const AppBarTheme(
284         backgroundColor: AppColors.surface,
285         foregroundColor: AppColors.onSurface,
286         elevation: 0,
287         centerTitle: false,
288         surfaceTintColor: Colors.transparent,
289     ),
290     cardTheme: CardThemeData(
291         color: cardColor,
292         elevation: 0,
293         surfaceTintColor: Colors.transparent,
294         shape: RoundedRectangleBorder(
295             borderRadius: BorderRadius.circular(AppRadius.xl),
296         ),
297     ),
298     elevatedButtonTheme: ElevatedButtonThemeData(
299         style: ElevatedButton.styleFrom(
300             backgroundColor: primaryColor,
```

第 07 页 第 301~350 行

```
301         foregroundColor: Colors.white,
302         minimumSize: const Size(0, 56),
303         padding: const EdgeInsets.symmetric(
304             horizontal: AppSpacing.xl,
305             vertical: AppSpacing.lg,
306     ),
```

```
307         elevation: 0,
308         shape: RoundedRectangleBorder(
309             borderRadius: BorderRadius.circular(AppRadius.lg),
310         ),
311     ),
312 ),
313 tabBarTheme: const TabBarThemeData(
314     dividerColor: Colors.transparent,
315     labelColor: AppColors.onSurface,
316     unselectedLabelColor: AppColors.onSurfaceVariant,
317     indicatorColor: AppColors.primary,
318 ),
319 snackBarTheme: SnackBarThemeData(
320     behavior: SnackBarBehavior.floating,
321     backgroundColor: AppColors.onSurface,
322     contentTextStyle: const TextStyle(color: Colors.white),
323     shape: RoundedRectangleBorder(
324         borderRadius: BorderRadius.circular(AppRadius.md),
325     ),
326 ),
327 inputDecorationTheme: InputDecorationTheme(
328     filled: true,
329     fillColor: AppColors.surfaceLowest,
330     contentPadding: const EdgeInsets.symmetric(
331         horizontal: AppSpacing.lg,
332         vertical: AppSpacing.lg,
333     ),
334     border: OutlineInputBorder(
335         borderRadius: BorderRadius.circular(AppRadius.lg),
336         borderSide: BorderSide.none,
337     ),
338     enabledBorder: OutlineInputBorder(
339         borderRadius: BorderRadius.circular(AppRadius.lg),
340         borderSide: BorderSide(color: Colors.grey.shade300),
341     ),
342     focusedBorder: OutlineInputBorder(
343         borderRadius: BorderRadius.circular(AppRadius.lg),
344         borderSide: const BorderSide(color: primaryColor),
345     ),
346 ),
347 );
348 }
349 }
350
```



```
352 // =====
353 // 文件: lib/core/audio/sound_player.dart
354 // =====
355 import 'package:flutter/foundation.dart';
356 import 'package:audioplayers/audioplayers.dart';
357
358 class SoundPlayer {
359   static final AudioPlayer _player = AudioPlayer();
360
361   static Future<void> playAsset(String path) async {
362     try {
363       await _player.stop();
364       await _player.play(AssetSource(path));
365     } catch (e) {
366       debugPrint('Error playing sound: $e');
367     }
368   }
369
370   static Future<void> playNewOrderAlert() async {
371     // This will play assets/audio/new_order.mp3
372     await playAsset('audio/new_order.mp3');
373   }
374
375   static void dispose() {
376     _player.dispose();
377   }
378 }
379
380
381 // =====
382 // 文件: lib/core/audio/tts_service.dart
383 // =====
384 import 'package:flutter/foundation.dart';
385 import 'package:flutter_tts/flutter_tts.dart';
386
387 class TtsService {
388   static final FlutterTts _flutterTts = FlutterTts();
389
390   static Future<void> init() async {
391     await _flutterTts.setLanguage("zh-CN");
392     await _flutterTts.setSpeechRate(0.5);
393     await _flutterTts.setVolume(1.0);
394     await _flutterTts.setPitch(1.0);
395   }
396
397   static Future<void> speak(String text) async {
398     try {
399       await _flutterTts.speak(text);
400     } catch (e) {
```

第 09 页 第 401~450 行

```
401     debugPrint('TTS error: $e');
402   }
403 }
404
405 static Future<void> speakOrderAlert(String orderNum, double amount) async {
406   final text = "订单 $orderNum 号, 金额 ${amount.toStringAsFixed(2)} 元";
407   await speak(text);
408 }
409 }
410
411
412 // =====
413 // 文件: lib/core/network/api_client.dart
414 // =====
415 import 'package:dio/dio.dart';
416 import 'package:flutter/foundation.dart';
417 import 'package:flutter_secure_storage/flutter_secure_storage.dart';
418 import 'package:merchant_app/config/env.dart';
419 import 'package:merchant_app/core/service/auth_session_controller.dart';
420
421 class ApiClient {
422   final Dio _dio;
423   final Dio _refreshDio;
424   final _storage = const FlutterSecureStorage();
425   final AuthSessionController _sessionController;
426   Future<String?>? _refreshFuture;
427
428   ApiClient(this._sessionController)
429     : _dio = Dio(BaseOptions(
430       baseUrl: Env.apiBaseUrl,
431       connectTimeout: const Duration(seconds: 10),
432       receiveTimeout: const Duration(seconds: 10),
433       contentType: 'application/json',
434     )),
435     _refreshDio = Dio(BaseOptions(
436       baseUrl: Env.apiBaseUrl,
437       connectTimeout: const Duration(seconds: 10),
438       receiveTimeout: const Duration(seconds: 10),
439       contentType: 'application/json',
440     )) {
441   if (kDebugMode) {
442     _dio.interceptors.add(LogInterceptor(
443       requestBody: false,
444       responseBody: false,
445     ));
```

```
446     }
447
448     _dio.interceptors.add(QueuedInterceptorsWrapper(
449       onRequest: (options, handler) async {
450         final requiresAuth = options.extra['requiresAuth'] != false;
```

第 10 页 第 451~500 行

```
451     if (!requiresAuth) {
452       return handler.next(options);
453     }
454
455     final accessToken = await _storage.read(key: 'access_token');
456     if (accessToken != null) {
457       options.headers['Authorization'] = 'Bearer $accessToken';
458     }
459     return handler.next(options);
460   },
461   onError: (e, handler) async {
462     final statusCode = e.response?.statusCode;
463     final requestOptions = e.requestOptions;
464     final isRefreshRequest = requestOptions.path == '/auth/refresh';
465     final alreadyRetried = requestOptions.extra['retriedAfterRefresh'] == true;
466     final requiresAuth = requestOptions.extra['requiresAuth'] != false;
467
468     if (statusCode == 401 && requiresAuth && !isRefreshRequest && !alreadyRetried) {
469       final refreshedAccessToken = await _refreshAccessToken();
470       if (refreshedAccessToken != null) {
471         requestOptions.headers['Authorization'] = 'Bearer $refreshedAccessToken';
472         requestOptions.extra['retriedAfterRefresh'] = true;
473
474         try {
475           final retryResponse = await _dio.fetch(requestOptions);
476           return handler.resolve(retryResponse);
477         } on DioException catch (retryError) {
478           return handler.next(retryError);
479         }
480       }
481     }
482
483     return handler.next(e);
484   },
485 ));
486 }
487
488 Future<String?> _refreshAccessToken() {
489   _refreshFuture ??= _performRefresh();
490   return _refreshFuture!.whenComplete(() => _refreshFuture = null);
```

```
491     }
492
493     Future<String?> _performRefresh() async {
494         final refreshToken = await _storage.read(key: 'refresh_token');
495         if (refreshToken == null || refreshToken.isEmpty) {
496             await _clearSession('缺少刷新令牌');
497             return null;
498         }
499
500         try {
```

第 11 页 第 501~550 行

```
501         final response = await _refreshDio.post(
502             '/auth/refresh',
503             data: {'refresh_token': refreshToken},
504         );
505
506         final data = response.data is Map<String, dynamic>
507             ? response.data as Map<String, dynamic>
508             : <String, dynamic>{};
509         final accessToken = data['access_token']?.toString();
510         final newRefreshToken = data['refresh_token']?.toString();
511
512         if (accessToken == null || newRefreshToken == null) {
513             await _clearSession('刷新登录状态失败');
514             return null;
515         }
516
517         await _storage.write(key: 'access_token', value: accessToken);
518         await _storage.write(key: 'refresh_token', value: newRefreshToken);
519         return accessToken;
520     } on DioException {
521         await _clearSession('登录状态已失效, 请重新绑定');
522         return null;
523     }
524 }
525
526 Future<void> _clearSession(String reason) async {
527     await _storage.delete(key: 'access_token');
528     await _storage.delete(key: 'refresh_token');
529     await _storage.delete(key: 'merchant_name');
530     _sessionController.invalidate(reason);
531 }
532
533 Future<Response> get(
534     String path, {
535     Map<String, dynamic>? queryParameters,
```

```
536     bool requiresAuth = true,
537   }) {
538     return _dio.get(
539       path,
540       queryParameters: queryParameters,
541       options: Options(extra: {'requiresAuth': requiresAuth}),
542     );
543   }
544
545   Future<Response> post(String path, {dynamic data, bool requiresAuth = true}) {
546     return _dio.post(
547       path,
548       data: data,
549       options: Options(extra: {'requiresAuth': requiresAuth}),
550     );
```

第 12 页 第 551~600 行

```
551   }
552
553   Future<Response> put(String path, {dynamic data, bool requiresAuth = true}) {
554     return _dio.put(
555       path,
556       data: data,
557       options: Options(extra: {'requiresAuth': requiresAuth}),
558     );
559   }
560
561   Future<Response> delete(String path, {bool requiresAuth = true}) {
562     return _dio.delete(
563       path,
564       options: Options(extra: {'requiresAuth': requiresAuth}),
565     );
566   }
567 }
568
569
570
571 // =====
572 // 文件: lib/core/network/api_provider.dart
573 // =====
574 import 'package:flutter_riverpod/flutter_riverpod.dart';
575 import 'package:merchant_app/core/network/api_client.dart';
576 import 'package:merchant_app/core/service/auth_session_controller.dart';
577
578 final authSessionControllerProvider = Provider<AuthSessionController>((ref) {
579   return AuthSessionController();
580 });
```

```
581
582 final apiClientProvider = Provider<ApiClient>((ref) {
583   final sessionController = ref.watch(authSessionControllerProvider);
584   return ApiClient(sessionController);
585 });
586
587
588 // =====
589 // 文件: lib/core/network/connectivity_provider.dart
590 // =====
591 import 'package:connectivity_plus/connectivity_plus.dart';
592 import 'package:flutter_riverpod/flutter_riverpod.dart';
593
594 final connectivityProvider = StreamProvider<bool>((ref) async* {
595   final connectivity = Connectivity();
596
597   // Initial check
598   final initialResult = await connectivity.checkConnectivity();
599   yield !initialResult.contains(ConnectivityResult.none);
600
```

第 13 页 第 601~650 行

```
601   // Yield stream updates
602   await for (final result in connectivity.onConnectivityChanged) {
603     yield !result.contains(ConnectivityResult.none);
604   }
605 });
606
607
608 // =====
609 // 文件: lib/core/network/ws_client.dart
610 // =====
611 import 'dart:async';
612 import 'dart:convert';
613 import 'dart:io';
614 import 'package:flutter/foundation.dart';
615 import 'package:web_socket_channel/io.dart';
616 import 'package:merchant_app/config/env.dart';
617 import 'package:merchant_app/core/service/message_dedup.dart';
618 import 'package:merchant_app/models/push_message.dart';
619
620 class WsClient {
621   final MessageDeduplicator _deduplicator;
622   IOWebSocketChannel? _channel;
623   Timer? _reconnectTimer;
624   bool _isDisposed = false;
625   bool _isConnecting = false;
```

```
626     int _reconnectAttempts = 0;
627     String? _currentToken;
628
629     void Function(PushMessage)? onNewOrder;
630     void Function(bool)? onStatusChange;
631
632     WsClient(this._deduplicator);
633
634     Future<void> connect(String token) async {
635         if (_isDisposed) return;
636         if (_isConnecting) return;
637         if (_channel != null && _currentToken == token) return;
638
639         _isConnecting = true;
640         _currentToken = token;
641         final uri = Uri.parse('${Env.wsUrl}?token=$token');
642         if (kDebugMode) {
643             debugPrint('Connecting to WebSocket: ${uri.replace(query: 'token=***')}');
644         }
645
646         try {
647             final socket = await WebSocket.connect(uri.toString());
648             socket.pingInterval = const Duration(seconds: 20);
649             _channel = IOWebSocketChannel(socket);
650             _reconnectAttempts = 0;
```

第 14 页 第 651~700 行

```
651         onStatusChange?.call(true);
652
653         _channel!.stream.listen(
654             (message) async {
655                 await _handleMessage(message);
656             },
657             onDone: () {
658                 if (kDebugMode) {
659                     debugPrint('WebSocket closed');
660                 }
661                 _channel = null;
662                 onStatusChange?.call(false);
663                 _scheduleReconnect(token);
664             },
665             onError: (error) {
666                 if (kDebugMode) {
667                     debugPrint('WebSocket error: $error');
668                 }
669                 _channel = null;
670                 onStatusChange?.call(false);
```

```
671         _scheduleReconnect(token);
672     },
673 );
674 } catch (e) {
675     if (kDebugMode) {
676         debugPrint('WebSocket connect error: $e');
677     }
678     _channel = null;
679     onStatusChange?.call(false);
680     _scheduleReconnect(token);
681 } finally {
682     _isConnecting = false;
683 }
684 }
685
686 Future<void> _handleMessage(dynamic message) async {
687     try {
688         final data = jsonDecode(message);
689         // Assuming the format from Go backend
690         if (data['type'] == 'order_notification' || data['type'] == 'notification') {
691             final payload = data['data'];
692             if (payload is! Map) {
693                 return;
694             }
695             final messageId = payload['message_id']?.toString();
696             final orderId = payload['order_id']?.toString();
697
698             if (messageId != null &&
699                 orderId != null &&
700                 await _deduplicator.tryAcceptGroup([
```

=====
第 15 页 第 701~750 行
=====

```
701         MessageDeduplicator.messageKey(messageId),
702         MessageDeduplicator.orderKey(orderId),
703     ])) {
704         final pushMsg = PushMessage.fromJson(Map<String, dynamic>.from(payload));
705         onNewOrder?.call(pushMsg);
706     }
707 }
708 } catch (e) {
709     if (kDebugMode) {
710         debugPrint('Error decoding WS message: $e');
711     }
712 }
713 }
714
715 void _scheduleReconnect(String token) {
```



```
716     _reconnectTimer?.cancel();
717     if (_isDisposed) return;
718
719     _reconnectAttempts += 1;
720     final delaySeconds = (_reconnectAttempts * 5).clamp(5, 30);
721
722     _reconnectTimer = Timer(Duration(seconds: delaySeconds), () {
723       connect(token);
724     });
725   }
726
727   void disconnect() {
728     _reconnectTimer?.cancel();
729     _reconnectAttempts = 0;
730     _currentToken = null;
731     _channel?.sink.close();
732     _channel = null;
733     onStatusChange?.call(false);
734   }
735
736   void dispose() {
737     _isDisposed = true;
738     disconnect();
739   }
740 }
741
742
743 // =====
744 // 文件: lib/core/network/ws_provider.dart
745 // =====
746 import 'package:flutter_riverpod/flutter_riverpod.dart';
747 import 'package:merchant_app/core/network/ws_client.dart';
748 import 'package:merchant_app/core/push/push_provider.dart';
749 import 'package:merchant_app/core/service/foreground_service.dart';
750 import 'package:merchant_app/core/network/connectivity_provider.dart';
```

=====

第 16 页 第 751~800 行

=====

```
751 import 'package:merchant_app/features/auth/auth_provider.dart';
752 import 'package:merchant_app/features/order/working_status_provider.dart';
753
754 final wsStatusProvider = StateProvider<bool>((ref) => false);
755
756 final wsClientProvider = Provider((ref) {
757   final deduplicator = ref.watch(messageDeduplicatorProvider);
758   final client = WsClient(deduplicator);
759
760   client.onStatusChange = (isConnected) {
```

```
761     ref.read(wsStatusProvider.notifier).state = isConnected;
762   };
763
764   // Clean up on dispose
765   ref.onDispose(() => client.dispose());
766
767   return client;
768 });
769
770 // Watcher to manage WS connection based on Auth state and Working Status
771 final wsConnectionManagerProvider = Provider((ref) {
772   final authState = ref.watch(authProvider);
773   final wsClient = ref.watch(wsClientProvider);
774   final isWorking = ref.watch(workingStatusProvider);
775   final hasNetwork = ref.watch(connectivityProvider).value ?? false;
776   final deviceSyncService = ref.watch(deviceSyncServiceProvider);
777
778   if (authState.isAuthenticated &&
779       authState.accessToken != null &&
780       isWorking &&
781       hasNetwork) {
782     wsClient.connect(authState.accessToken!);
783     MerchantForegroundService.start();
784     deviceSyncService.ensureRegistered();
785   } else {
786     wsClient.disconnect();
787     MerchantForegroundService.stop();
788   }
789
790   return null;
791 });
792
793
794 // =====
795 // 文件: lib/core/print/esc_pos_utils.dart
796 // =====
797 import 'dart:convert';
798 import 'package:merchant_app/models/push_message.dart';
799
800 class EscPosUtils {
```

=====

第 17 页 第 801~850 行

=====

```
801   static const List<int> _initPrinter = [27, 64];
802   static const List<int> _alignCenter = [27, 97, 1];
803   static const List<int> _alignLeft = [27, 97, 0];
804   static const List<int> _boldOn = [27, 69, 1];
805   static const List<int> _boldOff = [27, 69, 0];
```

```
806 static const List<int> _doubleHeightOn = [29, 33, 1];
807 static const List<int> _doubleHeightOff = [29, 33, 0];
808 static const List<int> _lineFeed = [10];
809
810 /// Generate raw ESC/POS bytes for an order receipt using UTF-8 or GBK.
811 /// Standard ESC/POS uses GBK for Chinese, but many modern cheap bluetooth
812 /// printers support UTF-8 if properly converted, or GB18030.
813 /// For simplicity, we assume UTF-8 is supported or it's a generic text transmission.
814 /// (In a production app, we would use a charset conversion library for GBK).
815 static List<int> generateOrderReceipt(PushMessage message) {
816     List<int> bytes = [];
817
818     // Init printer
819     bytes.addAll(_initPrinter);
820
821     // Title
822     bytes.addAll(_alignCenter);
823     bytes.addAll(_doubleHeightOn);
824     bytes.addAll(_boldOn);
825     bytes.addAll(_encode("乐客来福 外卖订单\n"));
826     bytes.addAll(_boldOff);
827     bytes.addAll(_doubleHeightOff);
828     bytes.addAll(_lineFeed);
829
830     // Order Info
831     bytes.addAll(_alignLeft);
832     bytes.addAll(_encode("所属店铺: ${message.shopName}\n"));
833     bytes.addAll(_encode("订单编号: ${message.displayOrderNumber}\n"));
834     bytes.addAll(_encode("-----\n"));
835
836     // Items (PushMessage currently doesn't carry items array, so we just show amount)
837     bytes.addAll(_boldOn);
838     bytes.addAll(_encode("订单总额: ¥${message.amount.toStringAsFixed(2)}\n"));
839     bytes.addAll(_boldOff);
840     bytes.addAll(_encode("-----\n"));
841
842     // Footer
843     bytes.addAll(_alignCenter);
844     bytes.addAll(_encode("感谢使用乐客来福商户版\n"));
845     bytes.addAll(_lineFeed);
846     bytes.addAll(_lineFeed);
847     bytes.addAll(_lineFeed);
848
849     return bytes;
850 }
```

```
851
852   static List<int> _encode(String text) {
853     return utf8.encode(text);
854   }
855 }
856
857
858 // =====
859 // 文件: lib/core/push/device_sync_service.dart
860 // =====
861 import 'dart:io';
862
863 import 'package:device_info_plus/device_info_plus.dart';
864 import 'package:dio/dio.dart';
865 import 'package:flutter/foundation.dart';
866 import 'package:flutter_secure_storage/flutter_secure_storage.dart';
867 import 'package:package_info_plus/package_info_plus.dart';
868
869 import 'package:merchant_app/core/network/api_client.dart';
870 import 'package:merchant_app/core/push/native_push_manager.dart';
871
872 class DeviceSyncService {
873   DeviceSyncService(this._apiClient, this._pushManager);
874
875   static const _lastRegisteredPushTokenKey = 'last_registered_push_token';
876   static const _deviceIdKey = 'device_uuid';
877
878   final ApiClient _apiClient;
879   final NativePushManager _pushManager;
880   final FlutterSecureStorage _storage = const FlutterSecureStorage();
881
882   Future<void>? _registrationFuture;
883   String? _lastRegisteredPushToken;
884
885   Future<void> ensureRegistered() {
886     _registrationFuture ??= _performEnsureRegistered()
887       .whenComplete(() => _registrationFuture = null);
888     return _registrationFuture!;
889   }
890
891   Future<void> _performEnsureRegistered() async {
892     final registrationId = await _pushManager.getRegistrationID();
893     if (registrationId == null || registrationId.isEmpty) {
894       return;
895     }
896
897     _lastRegisteredPushToken ??=
898       await _storage.read(key: _lastRegisteredPushTokenKey);
899     if (_lastRegisteredPushToken == registrationId) {
```

```
900         return;
```

```
=====
第 19 页 第 901~950 行
=====
```

```
901     }
902
903     final payload = await _buildDevicePayload(registrationId);
904
905     try {
906         await _apiClient.post('/merchant/device/register', data: payload);
907         await _storage.write(
908             key: _lastRegisteredPushTokenKey,
909             value: registrationId,
910         );
911         _lastRegisteredPushToken = registrationId;
912     } on DioException catch (error) {
913         if (_isEndpointNotReady(error)) {
914             if (kDebugMode) {
915                 debugPrint('Device register endpoint not ready yet.');
```

```
945     }
946   }
947 }
948
949 Future<Map<String, dynamic>> _buildDevicePayload(String registrationId) async {
950   final deviceInfo = DeviceInfoPlugin();
```

第 20 页 第 951~1000 行

```
951   var deviceModel = 'Unknown';
952   var osVersion = 'Unknown';
953
954   try {
955     if (Platform.isAndroid) {
956       final androidInfo = await deviceInfo.androidInfo;
957       deviceModel = androidInfo.model;
958       osVersion = 'Android ${androidInfo.version.release}';
959     } else if (Platform.isIOS) {
960       final iosInfo = await deviceInfo.iosInfo;
961       deviceModel = iosInfo.model;
962       osVersion = 'iOS ${iosInfo.systemVersion}';
963     }
964   } catch (_) {
965     if (kDebugMode) {
966       debugPrint('Failed to read device info for sync.');
```

```
990   }
991
992   Future<String> _getOrCreateDeviceId() async {
993     final existing = await _storage.read(key: _deviceIdKey);
994     if (existing != null && existing.isNotEmpty) {
995       return existing;
996     }
997
998     final fallback = DateTime.now().millisecondsSinceEpoch.toString();
999     await _storage.write(key: _deviceIdKey, value: fallback);
1000    return fallback;
```

第 21 页 第 1001~1050 行

```
1001   }
1002
1003   bool _isEndpointNotReady(DioException error) {
1004     final statusCode = error.response?.statusCode;
1005     return statusCode == 404 || statusCode == 405 || statusCode == 501;
1006   }
1007 }
1008
1009 // =====
1010 // 文件: lib/core/push/local_notification_service.dart
1011 // =====
1012 import 'dart:convert';
1013
1014 import 'package:flutter/foundation.dart';
1015 import 'package:flutter_local_notifications/flutter_local_notifications.dart';
1016 import 'package:merchant_app/models/push_message.dart';
1017 import 'package:permission_handler/permission_handler.dart';
1018
1019 class LocalNotificationService {
1020   LocalNotificationService();
1021
1022   final FlutterLocalNotificationsPlugin _plugin =
1023     FlutterLocalNotificationsPlugin();
1024
1025   bool _initialized = false;
1026
1027   void Function(PushMessage message)? onNotificationTap;
1028
1029   Future<void> init() async {
1030     if (_initialized) {
1031       return;
1032     }
1033
1034     const initializationSettings = InitializationSettings(
```

```
1035     android: AndroidInitializationSettings('@mipmap/launcher_icon'),
1036     iOS: DarwinInitializationSettings(),
1037 );
1038
1039 await _plugin.initialize(
1040     settings: initializationSettings,
1041     onDidReceiveNotificationResponse: _handleNotificationResponse,
1042     onDidReceiveBackgroundNotificationResponse: _handleBackgroundNotificationResponse,
1043 );
1044
1045 final androidPlugin =
1046     _plugin.resolvePlatformSpecificImplementation<
1047         AndroidFlutterLocalNotificationsPlugin>();
1048 if (androidPlugin != null) {
1049     await androidPlugin.createNotificationChannel(
1050         const AndroidNotificationChannel(
```

第 22 页 第 1051~1100 行

```
1051         'order_alert',
1052         '订单提醒',
1053         description: '新订单与超时提醒',
1054         importance: Importance.max,
1055         playSound: true,
1056     ),
1057 );
1058 await androidPlugin.createNotificationChannel(
1059     const AndroidNotificationChannel(
1060         'update_channel',
1061         '应用更新',
1062         description: '版本更新通知',
1063         importance: Importance.defaultImportance,
1064     ),
1065 );
1066 }
1067
1068 _initialized = true;
1069 }
1070
1071 Future<bool> ensureNotificationPermission() async {
1072     final status = await Permission.notification.status;
1073     if (status.isGranted) {
1074         return true;
1075     }
1076
1077     final result = await Permission.notification.request();
1078     return result.isGranted;
1079 }
```



```
1080
1081 Future<void> showNewOrderNotification(PushMessage message) async {
1082     await init();
1083
1084     const androidDetails = AndroidNotificationDetails(
1085         'order_alert',
1086         '订单提醒',
1087         channelDescription: '新订单与超时提醒',
1088         importance: Importance.max,
1089         priority: Priority.high,
1090         category: AndroidNotificationCategory.call,
1091         visibility: NotificationVisibility.public,
1092         fullScreenIntent: true,
1093         ticker: '新订单提醒',
1094     );
1095
1096     const notificationDetails = NotificationDetails(
1097         android: androidDetails,
1098         iOS: DarwinNotificationDetails(
1099             presentAlert: true,
1100             presentBadge: true,
```

=====

第 23 页 第 1101~1150 行

=====

```
1101         presentSound: true,
1102     ),
1103 );
1104
1105 await _plugin.show(
1106     id: message.notificationId,
1107     title: message.title.isNotEmpty ? message.title : '您有新的订单',
1108     body: message.content.isNotEmpty
1109         ? message.content
1110         : '订单号 ${message.displayOrderNumber}, 请及时处理',
1111     notificationDetails: notificationDetails,
1112     payload: jsonEncode(message.toJson()),
1113 );
1114 }
1115
1116 Future<void> _handleNotificationResponse(
1117     NotificationResponse response,
1118 ) async {
1119     final payload = response.payload;
1120     if (payload == null || payload.isEmpty) {
1121         return;
1122     }
1123
1124     try {
```

```
1125     final decoded = jsonDecode(payload);
1126     if (decoded is Map<String, dynamic>) {
1127         onNotificationTap?.call(PushMessage.fromJson(decoded));
1128     }
1129 } catch (error) {
1130     if (kDebugMode) {
1131         debugPrint('Failed to parse local notification payload: $error');
1132     }
1133 }
1134 }
1135 }
1136
1137 @pragma('vm:entry-point')
1138 void _handleBackgroundNotificationResponse(NotificationResponse response) {}
1139
1140 // =====
1141 // 文件: lib/core/push/native_push_manager.dart
1142 // =====
1143 import 'package:flutter/foundation.dart';
1144 import 'package:flutter/services.dart';
1145 import 'package:device_info_plus/device_info_plus.dart';
1146 import 'package:merchant_app/core/service/message_dedup.dart';
1147 import 'package:merchant_app/models/push_message.dart';
1148
1149 typedef PushOrderCallback = Future<void> Function(
1150     PushMessage message, {
```

第 24 页 第 1151~1200 行

```
1151     required bool showLocalNotification,
1152 });
1153
1154 class NativePushManager {
1155     static const MethodChannel _channel = MethodChannel('com.locallife.merchant/push');
1156
1157     final MessageDeduplicator _deduplicator;
1158
1159     // Callback when a valid new order arrives
1160     PushOrderCallback? onNewOrder;
1161     Future<void> Function(PushMessage message)? onNotificationOpened;
1162
1163     NativePushManager(this._deduplicator);
1164
1165     Future<void> init() async {
1166         _channel.setMethodCallHandler(_handleMethodCall);
1167
1168         // Initial native setup
1169         try {
```

```
1170     final String manufacturer = await _getManufacturer();
1171     await _channel.invokeMethod('initialize', {
1172         'manufacturer': manufacturer,
1173     });
1174     debugPrint('NativePushManager initialized for $manufacturer');
1175 } on PlatformException catch (e) {
1176     debugPrint('Failed to initialize native push: ${e.message}');
1177 }
1178 }
1179
1180 Future<String> _getManufacturer() async {
1181     final deviceInfo = DeviceInfoPlugin();
1182     if (defaultTargetPlatform == TargetPlatform.android) {
1183         final androidInfo = await deviceInfo.androidInfo;
1184         return androidInfo.manufacturer.toLowerCase();
1185     }
1186     return 'unknown';
1187 }
1188
1189 Future<void> _handleMethodCall(MethodCall call) async {
1190     switch (call.method) {
1191         case 'onReceiveMessage':
1192             final Map<String, dynamic> message = Map<String, dynamic>.from(call.arguments);
1193             await _handleIncomingMessage(message, showLocalNotification: true);
1194             break;
1195         case 'onNotificationOpened':
1196             final Map<String, dynamic> message = Map<String, dynamic>.from(call.arguments);
1197             final parsedMessage = _extractPushMessage(message);
1198             if (parsedMessage != null) {
1199                 await onNotificationOpened?.call(parsedMessage);
1200             }
1201         }
1202     }
```

第 25 页 第 1201~1250 行

```
1201         break;
1202     case 'onTokenRegistered':
1203         final String token = call.arguments['token'];
1204         final String provider = call.arguments['provider'];
1205         debugPrint('Push Token Registered: $token (Provider: $provider)');
1206         // TODO: Upload to backend
1207         break;
1208     }
1209 }
1210
1211 Future<void> _handleIncomingMessage(
1212     Map<String, dynamic> message, {
1213     required bool showLocalNotification,
1214 }) async {
```

```
1215     final pushMessage = _extractPushMessage(message);
1216     if (pushMessage == null) return;
1217
1218     final accepted = await _deduplicator.tryAcceptGroup([
1219       MessageDeduplicator.messageKey(pushMessage.messageId),
1220       MessageDeduplicator.orderKey(pushMessage.orderId),
1221     ]);
1222
1223     if (accepted) {
1224       await onNewOrder?.call(
1225         pushMessage,
1226         showLocalNotification: showLocalNotification,
1227       );
1228     }
1229   }
1230
1231   PushMessage? _extractPushMessage(Map<String, dynamic> message) {
1232     // Standardize payload extraction from different vendors if needed
1233     // Assuming native side sends a clean map
1234     try {
1235       return PushMessage.fromJson(message);
1236     } catch (e) {
1237       debugPrint('Error parsing push message: $e');
1238       return null;
1239     }
1240   }
1241
1242   Future<String?> getRegistrationID() async {
1243     return await _channel.invokeMethod<String>('getRegistrationId');
1244   }
1245 }
1246
1247
1248 // =====
1249 // 文件: lib/core/push/push_provider.dart
1250 // =====
```

第 26 页 第 1251~1300 行

```
1251 import 'package:flutter_riverpod/flutter_riverpod.dart';
1252 import 'package:merchant_app/core/network/api_provider.dart';
1253 import 'package:merchant_app/core/push/device_sync_service.dart';
1254 import 'package:merchant_app/core/service/message_dedup.dart';
1255 import 'package:merchant_app/core/push/native_push_manager.dart';
1256
1257 final messageDeduplicatorProvider = Provider((ref) => MessageDeduplicator());
1258
1259 final pushManagerProvider = Provider((ref) {
```

```
1260   final deduplicator = ref.watch(messageDeduplicatorProvider);
1261   return NativePushManager(deduplicator);
1262 });
1263
1264 final deviceSyncServiceProvider = Provider<DeviceSyncService>((ref) {
1265   final apiClient = ref.watch(apiClientProvider);
1266   final pushManager = ref.watch(pushManagerProvider);
1267   return DeviceSyncService(apiClient, pushManager);
1268 });
1269
1270
1271 // =====
1272 // 文件: lib/core/service/auth_session_controller.dart
1273 // =====
1274 import 'package:flutter/foundation.dart';
1275
1276 class AuthSessionController extends ChangeNotifier {
1277   String? _lastInvalidationReason;
1278
1279   String? get lastInvalidationReason => _lastInvalidationReason;
1280
1281   void invalidate([String? reason]) {
1282     _lastInvalidationReason = reason;
1283     notifyListeners();
1284   }
1285
1286   void clearInvalidation() {
1287     _lastInvalidationReason = null;
1288   }
1289 }
1290
1291 // =====
1292 // 文件: lib/core/service/foreground_service.dart
1293 // =====
1294 import 'package:flutter/foundation.dart';
1295 import 'package:flutter_foreground_task/flutter_foreground_task.dart';
1296
1297 class MerchantForegroundService {
1298   static Future<void> init() async {
1299     FlutterForegroundTask.init(
1300       androidNotificationOptions: AndroidNotificationOptions(
```

第 27 页 第 1301~1350 行

```
1301     channelId: 'merchant_fg_service',
1302     channelName: '商户在线服务',
1303     channelDescription: '保持商户型在线接收订单',
1304     channelImportance: NotificationChannelImportance.LOW,
```

```
1305     priority: NotificationPriority.LOW,
1306   ),
1307   iosNotificationOptions: const IOSNotificationOptions(),
1308   foregroundTaskOptions: ForegroundTaskOptions(
1309     eventAction: ForegroundTaskEventAction.repeat(15000),
1310     autoRunOnBoot: true,
1311     allowWakeLock: true,
1312     allowWifiLock: true,
1313   ),
1314 );
1315 }
1316
1317 static Future<void> start() async {
1318   if (await FlutterForegroundTask.isRunningService) {
1319     await FlutterForegroundTask.restartService();
1320   } else {
1321     await FlutterForegroundTask.startService(
1322       notificationTitle: '乐客来福商户端',
1323       notificationText: '后台运行中 • 等待新订单...',
1324       callback: startCallback,
1325     );
1326   }
1327 }
1328
1329 static Future<void> stop() async {
1330   await FlutterForegroundTask.stopService();
1331 }
1332 }
1333
1334 @pragma('vm:entry-point')
1335 void startCallback() {
1336   FlutterForegroundTask.setTaskHandler(MerchantTaskHandler());
1337 }
1338
1339 class MerchantTaskHandler extends TaskHandler {
1340   @override
1341   Future<void> onStart(DateTime timestamp, TaskStarter starter) async {
1342     if (kDebugMode) {
1343       debugPrint('Foreground task started at $timestamp by $starter');
1344     }
1345   }
1346
1347   @override
1348   void onRepeatEvent(DateTime timestamp) {
1349     // Heartbeat logic or polling fallback can be added here
1350     if (kDebugMode) {
```

```
=====
1351     debugPrint('Foreground service heartbeat at $timestamp');
1352   }
1353 }
1354
1355 @override
1356 Future<void> onDestroy(DateTime timestamp, bool isTimeout) async {
1357   if (kDebugMode) {
1358     debugPrint('Foreground service destroyed (isTimeout: $isTimeout)');
1359   }
1360 }
1361 }
1362
1363
1364 // =====
1365 // 文件: lib/core/service/message_dedup.dart
1366 // =====
1367
1368 import 'package:path/path.dart' as path;
1369 import 'package:sqflite/sqflite.dart';
1370
1371 class MessageDeduplicator {
1372   static const _maxMemorySize = 500;
1373   static const _retention = Duration(hours: 24);
1374
1375   final _memoryCache = <String, DateTime>{};
1376   Database? _database;
1377   Future<void>? _initFuture;
1378
1379   Future<void> ensureInitialized() {
1380     _initFuture ??= _initialize();
1381     return _initFuture!;
1382   }
1383
1384   Future<void> _initialize() async {
1385     final databasesPath = await getDatabasesPath();
1386     final databasePath = path.join(databasesPath, 'message_dedup.db');
1387     _database = await openDatabase(
1388       databasePath,
1389       version: 1,
1390       onCreate: (db, version) async {
1391         await db.execute('''
1392           CREATE TABLE processed_messages (
1393             message_id TEXT PRIMARY KEY,
1394             processed_at INTEGER NOT NULL
1395           )
1396         ''');
1397       },
1398     );
1399   }
1400 }
```

```
1399     await _cleanupExpired();
1400 }
```

第 29 页 第 1401~1450 行

```
1401
1402 Future<bool> tryAccept(String messageId) async {
1403     return tryAcceptGroup([messageId]);
1404 }
1405
1406 Future<bool> tryAcceptGroup(List<String> rawKeys) async {
1407     await ensureInitialized();
1408
1409     final keys = rawKeys
1410         .map((key) => key.trim())
1411         .where((key) => key.isNotEmpty)
1412         .toSet()
1413         .toList(growable: false);
1414     if (keys.isEmpty) {
1415         return false;
1416     }
1417
1418     for (final key in keys) {
1419         if (_memoryCache.containsKey(key)) {
1420             return false;
1421         }
1422     }
1423
1424     final now = DateTime.now();
1425     final cutoff = now.subtract(_retention).millisecondsSinceEpoch;
1426     final db = _database;
1427     if (db == null) {
1428         return false;
1429     }
1430
1431     final placeholders = List.filled(keys.length, '?').join(', ');
1432     final existing = await db.query(
1433         'processed_messages',
1434         columns: ['message_id'],
1435         where: 'message_id IN ($placeholders) AND processed_at >= ?',
1436         whereArgs: [...keys, cutoff],
1437         limit: 1,
1438     );
1439     if (existing.isNotEmpty) {
1440         return false;
1441     }
1442
1443     final batch = db.batch();
```



```
1444     for (final key in keys) {
1445         _remember(key, now);
1446         batch.insert(
1447             'processed_messages',
1448             {
1449                 'message_id': key,
1450                 'processed_at': now.millisecondsSinceEpoch,
```

第 30 页 第 1451~1500 行

```
1451         },
1452         conflictAlgorithm: ConflictAlgorithm.ignore,
1453     );
1454 }
1455 await batch.commit(noResult: true);
1456
1457 if (_memoryCache.length % 50 == 0) {
1458     await _cleanupExpired();
1459 }
1460
1461 return true;
1462 }
1463
1464 static String messageKey(String messageId) => 'message:$messageId';
1465
1466 static String orderKey(String orderId) => 'order:$orderId';
1467
1468 void _remember(String messageId, DateTime timestamp) {
1469     _memoryCache[messageId] = timestamp;
1470     if (_memoryCache.length > _maxMemorySize) {
1471         final oldestKey = _memoryCache.entries
1472             .reduce((left, right) => left.value.isBefore(right.value) ? left : right)
1473             .key;
1474         _memoryCache.remove(oldestKey);
1475     }
1476 }
1477
1478 Future<void> _cleanupExpired() async {
1479     final db = _database;
1480     if (db == null) {
1481         return;
1482     }
1483
1484     final cutoff = DateTime.now().subtract(_retention).millisecondsSinceEpoch;
1485     await db.delete(
1486         'processed_messages',
1487         where: 'processed_at < ?',
1488         whereArgs: [cutoff],
```

```
1489     );
1490     _memoryCache.removeWhere((_, processedAt) => processedAt.millisecondsSinceEpoch < cutoff);
1491   }
1492 }
1493
1494
1495 // =====
1496 // 文件: lib/core/service/navigation_service.dart
1497 // =====
1498 import 'package:flutter/material.dart';
1499
1500 final GlobalKey<NavigatorState> rootNavigatorKey = GlobalKey<NavigatorState>();
```

【后段源程序（第 31 页 ~ 第 60 页，第 4512 行 ~ 第 6011 行）】

第 31 页 第 4512~4561 行

```
4512         style: TextStyle(fontSize: 24, fontWeight: FontWeight.w700),
4513       ),
4514       const SizedBox(height: AppSpacing.xs),
4515       packageInfoAsync.when(
4516         data: (info) => Text(
4517           '版本 ${info.version}',
4518           style: const TextStyle(
4519             fontSize: 15,
4520             color: AppColors.onSurfaceVariant,
4521           ),
4522         ),
4523       loading: () => const SizedBox(
4524         height: 20,
4525         width: 20,
4526         child: CircularProgressIndicator(strokeWidth: 2),
4527       ),
4528       error: (error, stackTrace) => const Text('获取版本失败'),
4529     ),
4530   ],
4531 ),
4532 ),
4533 ),
4534 const SizedBox(height: AppSpacing.lg),
4535 Card(
4536   child: Padding(
4537     padding: const EdgeInsets.all(AppSpacing.xl),
4538     child: Column(
4539       children: [
```

```
4540         _AboutActionTile(  
4541             icon: Icons.system_update_alt,  
4542             title: '检查更新',  
4543             subtitle: '检查当前版本是否有可用新版本。',  
4544             onTap: () {  
4545                 ref  
4546                     .read(updateServiceProvider)  
4547                     .checkForUpdates(context, showToastIfLatest: true);  
4548             },  
4549         ),  
4550         const SizedBox(height: AppSpacing.md),  
4551         const _AboutActionTile(  
4552             icon: Icons.privacy_tip_outlined,  
4553             title: '隐私政策',  
4554             subtitle: '查看当前生效的隐私政策与个人信息处理规则。',  
4555             route: '/settings/agreement/PRIVACY_POLICY',  
4556         ),  
4557         const SizedBox(height: AppSpacing.md),  
4558         const _AboutActionTile(  
4559             icon: Icons.description_outlined,  
4560             title: '用户协议',  
4561             subtitle: '查看当前生效的用户协议。',
```

=====
第 32 页 第 4562~4611 行
=====

```
4562             route: '/settings/agreement/USER_AGREEMENT',  
4563         ),  
4564     ],  
4565 ),  
4566 ),  
4567 ),  
4568 const SizedBox(height: AppSpacing.xl),  
4569 const Center(  
4570     child: Text(  
4571         '© 2026 乐客来福科技 版权所有',  
4572         style: TextStyle(  
4573             color: AppColors.onSurfaceVariant,  
4574             fontSize: 12,  
4575         ),  
4576     ),  
4577 ),  
4578 const SizedBox(height: AppSpacing.xl),  
4579 ],  
4580 ),  
4581 ),  
4582 ),  
4583 );  
4584 }
```

```
4585 }
4586
4587 class _AboutActionTile extends StatelessWidget {
4588   const _AboutActionTile({
4589     required this.icon,
4590     required this.title,
4591     required this.subtitle,
4592     this.onTap,
4593     this.route,
4594   });
4595
4596   final IconData icon;
4597   final String title;
4598   final String subtitle;
4599   final VoidCallback? onTap;
4600   final String? route;
4601
4602   @override
4603   Widget build(BuildContext context) {
4604     final effectiveTap = onTap ?? (route == null ? null : () => context.push(route!));
4605     final content = Container(
4606       padding: const EdgeInsets.all(AppSpacing.lg),
4607       decoration: BoxDecoration(
4608         color: AppColors.surfaceLow,
4609         borderRadius: BorderRadius.circular(AppRadius.lg),
4610       ),
4611       child: Row(
```

第 33 页 第 4612~4661 行

```
4612       children: [
4613         Icon(icon, color: AppColors.primary),
4614         const SizedBox(width: AppSpacing.md),
4615         Expanded(
4616           child: Column(
4617             crossAxisAlignment: CrossAxisAlignment.start,
4618             children: [
4619               Text(
4620                 title,
4621                 style: const TextStyle(
4622                   fontWeight: FontWeight.w700,
4623                 ),
4624             ),
4625             const SizedBox(height: AppSpacing.xs),
4626             Text(
4627               subtitle,
4628               style: const TextStyle(
4629                 color: AppColors.onSurfaceVariant,
```

```
4630             height: 1.45,
4631           ),
4632         ),
4633       ],
4634     ),
4635   ),
4636   const SizedBox(width: AppSpacing.md),
4637   Icon(
4638     effectiveTap == null ? Icons.schedule_outlined : Icons.chevron_right,
4639     color: AppColors.onSurfaceVariant,
4640   ),
4641 ],
4642 ),
4643 );
4644
4645 if (effectiveTap == null) {
4646   return content;
4647 }
4648
4649 return InkWell(
4650   onTap: effectiveTap,
4651   borderRadius: BorderRadius.circular(AppRadius.lg),
4652   child: content,
4653 );
4654 }
4655 }
4656
4657
4658 // =====
4659 // 文件: lib/features/settings/agreement_page.dart
4660 // =====
4661 import 'package:flutter/material.dart';
```

第 34 页 第 4662~4711 行

```
4662 import 'package:flutter_riverpod/flutter_riverpod.dart';
4663 import 'package:merchant_app/config/theme.dart';
4664 import 'package:merchant_app/core/network/api_provider.dart';
4665 import 'package:merchant_app/widgets/merchant_content_shell.dart';
4666
4667 class AgreementDetail {
4668   const AgreementDetail({
4669     required this.type,
4670     required this.title,
4671     required this.version,
4672     required this.publishedOn,
4673     required this.content,
4674   });
```

```
4675
4676     final String type;
4677     final String title;
4678     final String version;
4679     final String publishedOn;
4680     final String content;
4681
4682     factory AgreementDetail.fromJson(Map<String, dynamic> json) {
4683         return AgreementDetail(
4684             type: json['type']?.toString() ?? '',
4685             title: json['title']?.toString() ?? '',
4686             version: json['version']?.toString() ?? '',
4687             publishedOn: json['published_on']?.toString() ?? '',
4688             content: json['content']?.toString() ?? '',
4689         );
4690     }
4691 }
4692
4693 final agreementProvider =
4694     FutureProvider.family<AgreementDetail, String>((ref, type) async {
4695         final apiClient = ref.watch(apiClientProvider);
4696         final response = await apiClient.get('/agreements/$type');
4697         return AgreementDetail.fromJson(Map<String, dynamic>.from(response.data));
4698     });
4699
4700 class AgreementPage extends ConsumerWidget {
4701     const AgreementPage({
4702         required this.agreementType,
4703         super.key,
4704     });
4705
4706     final String agreementType;
4707
4708     @override
4709     Widget build(BuildContext context, WidgetRef ref) {
4710         final agreementAsync = ref.watch(agreementProvider(agreementType));
4711
```

第 35 页 第 4712~4761 行

```
4712     return Scaffold(
4713         appBar: AppBar(
4714             title: const Text('协议详情'),
4715         ),
4716         body: agreementAsync.when(
4717             data: (agreement) => SingleChildScrollView(
4718                 child: MerchantContentShell(
4719                     child: Column(
```

```
4720         crossAxisAlignment: CrossAxisAlignment.stretch,
4721         children: [
4722           Card(
4723             child: Padding(
4724               padding: const EdgeInsets.all(AppSpacing.xl),
4725               child: Column(
4726                 crossAxisAlignment: CrossAxisAlignment.start,
4727                 children: [
4728                   Text(
4729                     agreement.title,
4730                     style: const TextStyle(
4731                       fontSize: 22,
4732                       fontWeight: FontWeight.w700,
4733                     ),
4734                   ),
4735                   const SizedBox(height: AppSpacing.sm),
4736                   Text(
4737                     '版本 ${agreement.version} · 发布于 ${agreement.publishedOn}',
4738                     style: const TextStyle(
4739                       color: AppColors.onSurfaceVariant,
4740                       height: 1.45,
4741                     ),
4742                   ),
4743                 ],
4744             ),
4745           ),
4746         ),
4747         const SizedBox(height: AppSpacing.lg),
4748         Card(
4749           child: Padding(
4750             padding: const EdgeInsets.all(AppSpacing.xl),
4751             child: SelectableText(
4752               _htmlToReadableText(agreement.content),
4753               style: const TextStyle(
4754                 fontSize: 15,
4755                 height: 1.7,
4756                 color: AppColors.onSurface,
4757               ),
4758             ),
4759           ),
4760         ),
4761         const SizedBox(height: AppSpacing.xl),
```

第 36 页 第 4762~4811 行

```
4762         ],
4763       ),
4764     ),
```

```
4765     ),
4766     loading: () => const Center(child: CircularProgressIndicator()),
4767     error: (error, stackTrace) => Center(
4768       child: Padding(
4769         padding: const EdgeInsets.all(AppSpacing.xl),
4770         child: Column(
4771           mainAxisAlignment: MainAxisAlignment.min,
4772           children: [
4773             const Icon(
4774               Icons.description_outlined,
4775               size: 48,
4776               color: AppColors.onSurfaceVariant,
4777             ),
4778             const SizedBox(height: AppSpacing.lg),
4779             const Text(
4780               '协议内容加载失败',
4781               style: TextStyle(
4782                 fontSize: 18,
4783                 fontWeight: FontWeight.w700,
4784               ),
4785             ),
4786             const SizedBox(height: AppSpacing.sm),
4787             Text(
4788               error.toString(),
4789               textAlign: TextAlign.center,
4790               style: const TextStyle(
4791                 color: AppColors.onSurfaceVariant,
4792                 height: 1.45,
4793               ),
4794             ),
4795           ],
4796         ),
4797       ),
4798     ),
4799   ),
4800 );
4801 }
4802
4803 static String _htmlToReadableText(String html) {
4804   var text = html
4805     .replaceAll(RegExp(r'<style[\s\S]*?</style>', caseSensitive: false), '')
4806     .replaceAll(RegExp(r'<script[\s\S]*?</script>', caseSensitive: false), '')
4807     .replaceAll(RegExp(r'<br\s*/?>', caseSensitive: false), '\n')
4808     .replaceAll(RegExp(r'</p>|</div>|</h1>|</h2>|</h3>|</li>', caseSensitive: false), '\n')
4809     .replaceAll(RegExp(r'<li[^>]*>', caseSensitive: false), '• ')
4810     .replaceAll(RegExp(r'<[\>]+>', ''))
4811     .replaceAll('&nbsp;', ' ')
```

=====


```
=====
4812      .replaceAll('&', '&')
4813      .replaceAll('<', '<')
4814      .replaceAll('>', '>')
4815      .replaceAll('#39;', "'")
4816      .replaceAll('"', '"');
4817
4818      text = text.replaceAll(RegExp(r'\n\s*\n\s*\n+'), '\n\n');
4819      return text.trim();
4820  }
4821 }
4822
4823 // =====
4824 // 文件: lib/features/settings/notification_settings_provider.dart
4825 // =====
4826 import 'package:flutter_riverpod/flutter_riverpod.dart';
4827 import 'package:shared_preferences/shared_preferences.dart';
4828
4829 class NotificationSettingsState {
4830   const NotificationSettingsState({
4831     this.soundEnabled = true,
4832     this.voiceEnabled = true,
4833     this.autoAcceptEnabled = false,
4834     this.autoPrintAfterAcceptEnabled = true,
4835   });
4836
4837   final bool soundEnabled;
4838   final bool voiceEnabled;
4839   final bool autoAcceptEnabled;
4840   final bool autoPrintAfterAcceptEnabled;
4841
4842   NotificationSettingsState copyWith({
4843     bool? soundEnabled,
4844     bool? voiceEnabled,
4845     bool? autoAcceptEnabled,
4846     bool? autoPrintAfterAcceptEnabled,
4847   }) {
4848     return NotificationSettingsState(
4849       soundEnabled: soundEnabled ?? this.soundEnabled,
4850       voiceEnabled: voiceEnabled ?? this.voiceEnabled,
4851       autoAcceptEnabled: autoAcceptEnabled ?? this.autoAcceptEnabled,
4852       autoPrintAfterAcceptEnabled:
4853         autoPrintAfterAcceptEnabled ?? this.autoPrintAfterAcceptEnabled,
4854     );
4855   }
4856 }
4857
4858 class NotificationSettingsNotifier
```

```
4859     extends StateNotifier<NotificationSettingsState> {
4860     NotificationSettingsNotifier() : super(const NotificationSettingsState()) {
4861         _load();
```

=====
第 38 页 第 4862~4911 行
=====

```
4862     }
4863
4864     static const _soundKey = 'notification_sound_enabled';
4865     static const _voiceKey = 'notification_voice_enabled';
4866     static const _autoAcceptKey = 'order_auto_accept_enabled';
4867     static const _autoPrintAfterAcceptKey = 'order_auto_print_after_accept_enabled';
4868
4869     Future<void> _load() async {
4870         final prefs = await SharedPreferences.getInstance();
4871         state = state.copyWith(
4872             soundEnabled: prefs.getBool(_soundKey) ?? true,
4873             voiceEnabled: prefs.getBool(_voiceKey) ?? true,
4874             autoAcceptEnabled: prefs.getBool(_autoAcceptKey) ?? false,
4875             autoPrintAfterAcceptEnabled:
4876                 prefs.getBool(_autoPrintAfterAcceptKey) ?? true,
4877         );
4878     }
4879
4880     Future<void> setSoundEnabled(bool enabled) async {
4881         state = state.copyWith(soundEnabled: enabled);
4882         final prefs = await SharedPreferences.getInstance();
4883         await prefs.setBool(_soundKey, enabled);
4884     }
4885
4886     Future<void> setVoiceEnabled(bool enabled) async {
4887         state = state.copyWith(voiceEnabled: enabled);
4888         final prefs = await SharedPreferences.getInstance();
4889         await prefs.setBool(_voiceKey, enabled);
4890     }
4891
4892     Future<void> setAutoAcceptEnabled(bool enabled) async {
4893         state = state.copyWith(autoAcceptEnabled: enabled);
4894         final prefs = await SharedPreferences.getInstance();
4895         await prefs.setBool(_autoAcceptKey, enabled);
4896     }
4897
4898     Future<void> setAutoPrintAfterAcceptEnabled(bool enabled) async {
4899         state = state.copyWith(autoPrintAfterAcceptEnabled: enabled);
4900         final prefs = await SharedPreferences.getInstance();
4901         await prefs.setBool(_autoPrintAfterAcceptKey, enabled);
4902     }
4903 }
```

```
4904
4905 final notificationSettingsProvider = StateNotifierProvider<
4906     NotificationSettingsNotifier, NotificationSettingsState>((ref) {
4907     return NotificationSettingsNotifier();
4908 });
4909
4910 // =====
4911 // 文件: lib/features/settings/permission_guide_page.dart
```

=====

第 39 页 第 4912~4961 行

=====

```
4912 // =====
4913 import 'package:flutter/material.dart';
4914 import 'package:app_settings/app_settings.dart';
4915 import 'package:merchant_app/config/theme.dart';
4916 import 'package:merchant_app/widgets/merchant_content_shell.dart';
4917 import 'package:merchant_app/widgets/merchant_primary_button.dart';
4918
4919 class PermissionGuidePage extends StatelessWidget {
4920     const PermissionGuidePage({super.key});
4921
4922     @override
4923     Widget build(BuildContext context) {
4924         return Scaffold(
4925             appBar: AppBar(
4926                 title: const Text('自启动与保活设置'),
4927             ),
4928             body: SingleChildScrollView(
4929                 child: MerchantContentShell(
4930                     child: Column(
4931                         children: [
4932                             Container(
4933                                 padding: const EdgeInsets.all(AppSpacing.xl),
4934                                 decoration: BoxDecoration(
4935                                     color: AppColors.warningSoft,
4936                                     borderRadius: BorderRadius.circular(AppRadius.xl),
4937                                 ),
4938                                 child: const Row(
4939                                     crossAxisAlignment: CrossAxisAlignment.start,
4940                                     children: [
4941                                         Icon(
4942                                             Icons.warning_amber_rounded,
4943                                             color: AppColors.secondary,
4944                                         ),
4945                                         SizedBox(width: AppSpacing.md),
4946                                         Expanded(
4947                                             child: Text(
4948                                                 '为保证息屏或退到后台后仍能收到订单提醒, 请按手机品牌完成以下保活设置。',
```

```
4949         style: TextStyle(
4950             color: AppColors.onSurface,
4951             height: 1.5,
4952         ),
4953     ),
4954 ),
4955 ],
4956 ),
4957 ),
4958 const SizedBox(height: AppSpacing.lg),
4959 _buildBrandGuide(
4960     brand: '华为 / 荣耀 (EMUI / HarmonyOS)',
4961     steps: [
```

=====
第 40 页 第 4962~5011 行
=====

```
4962         '1. 在应用启动管理中关闭自动管理，并手动开启自启动、关联启动、后台活动。',
4963         '2. 在电池设置中开启休眠时保持网络连接。',
4964         '3. 将乐客来福加入不受电池优化限制名单。',
4965     ],
4966 ),
4967 _buildBrandGuide(
4968     brand: '小米 / Redmi (MIUI / HyperOS)',
4969     steps: [
4970         '1. 在应用管理中开启乐客来福自启动。',
4971         '2. 在省电与电池设置中改为无限制。',
4972         '3. 在最近任务界面锁定乐客来福卡片，避免被系统回收。',
4973     ],
4974 ),
4975 _buildBrandGuide(
4976     brand: 'OPPO / Realme (ColorOS)',
4977     steps: [
4978         '1. 在耗电保护中开启允许后台运行。',
4979         '2. 在自启动管理中允许乐客来福自动启动。',
4980         '3. 允许通知与状态栏提醒，避免新订单无提示。',
4981     ],
4982 ),
4983 _buildBrandGuide(
4984     brand: 'vivo (OriginOS / Funtouch)',
4985     steps: [
4986         '1. 在后台耗电管理中允许后台高耗电。',
4987         '2. 在权限管理中开启乐客来福自启动。',
4988     ],
4989 ),
4990 const SizedBox(height: AppSpacing.xl),
4991 MerchantPrimaryButton(
4992     label: '前往系统设置',
4993     onPressed: () {
```

```
4994         AppSettings.openAppSettings();
4995     },
4996     icon: const Icon(Icons.settings),
4997 ),
4998     const SizedBox(height: AppSpacing.xl),
4999 ],
5000 ),
5001 ),
5002 ),
5003 );
5004 }
5005
5006 Widget _buildBrandGuide({required String brand, required List<String> steps}) {
5007     return Card(
5008         margin: const EdgeInsets.symmetric(vertical: AppSpacing.sm),
5009         child: ExpansionTile(
5010             title: Text(
5011                 brand,
```

=====
第 41 页 第 5012~5061 行
=====

```
5012         style: const TextStyle(fontWeight: FontWeight.w700),
5013     ),
5014     leading: const Icon(Icons.phone_android, color: AppColors.primary),
5015     children: [
5016         Padding(
5017             padding: const EdgeInsets.only(
5018                 left: AppSpacing.lg,
5019                 right: AppSpacing.lg,
5020                 bottom: AppSpacing.lg,
5021             ),
5022             child: Column(
5023                 crossAxisAlignment: CrossAxisAlignment.start,
5024                 children: steps
5025                     .map((step) => Padding(
5026                         padding: const EdgeInsets.only(top: AppSpacing.sm),
5027                         child: Text(
5028                             step,
5029                             style: const TextStyle(fontSize: 14, height: 1.5),
5030                         ),
5031                     ))
5032                 .toList(),
5033         ),
5034     ),
5035 ],
5036 ),
5037 );
5038 }
```

```
5039 }
5040
5041
5042 // =====
5043 // 文件: lib/features/settings/settings_page.dart
5044 // =====
5045 import 'package:flutter/material.dart';
5046 import 'package:flutter_riverpod/flutter_riverpod.dart';
5047 import 'package:go_router/go_router.dart';
5048 import 'package:merchant_app/config/theme.dart';
5049 import 'package:merchant_app/features/auth/auth_provider.dart';
5050 import 'package:merchant_app/features/order/order_provider.dart';
5051 import 'package:merchant_app/features/order/working_status_provider.dart';
5052 import 'package:merchant_app/features/printer/printer_provider.dart';
5053 import 'package:merchant_app/features/settings/notification_settings_provider.dart';
5054 import 'package:merchant_app/widgets/merchant_content_shell.dart';
5055 import 'package:merchant_app/widgets/merchant_status_badge.dart';
5056
5057 class SettingsPage extends ConsumerWidget {
5058   const SettingsPage({super.key});
5059
5060   @override
5061   Widget build(BuildContext context, WidgetRef ref) {
```

=====

第 42 页 第 5062~5111 行

=====

```
5062     final isAuthenticated = ref.watch(authProvider.select((state) => state.isAuthenticated));
5063     final printerState = ref.watch(printerProvider);
5064     final notificationSettings = ref.watch(notificationSettingsProvider);
5065     final notificationSettingsNotifier =
5066       ref.read(notificationSettingsProvider.notifier);
5067
5068     return Scaffold(
5069       appBar: AppBar(
5070         title: const Text('系统设置'),
5071       ),
5072       body: ListView(
5073         padding: EdgeInsets.zero,
5074         children: [
5075           MerchantContentShell(
5076             child: Column(
5077               crossAxisAlignment: CrossAxisAlignment.stretch,
5078               children: [
5079                 _buildSectionCard(
5080                   title: '硬件与连接',
5081                   description: '查看当前打印与消息接收环境是否可用。',
5082                   children: [
5083                     _buildActionTile(
```

```
5084         context: context,
5085         icon: Icons.print_outlined,
5086         title: '小票打印机',
5087         subtitle: printerState.connectedDevice != null
5088             ? '已连接 ${printerState.connectedDevice!.platformName}'
5089             : '未连接蓝牙打印机',
5090         badge: MerchantStatusBadge(
5091             label: printerState.connectedDevice != null ? '已连接' : '未连接',
5092             tone: printerState.connectedDevice != null
5093                 ? MerchantStatusTone.positive
5094                 : MerchantStatusTone.neutral,
5095         ),
5096         onTap: () => context.push('/printer'),
5097     ),
5098 ],
5099 ),
5100 const SizedBox(height: AppSpacing.lg),
5101 _buildSectionCard(
5102     title: '提醒与保活',
5103     description: '设置来单提醒、接单方式和接单后打印, 并检查系统保活状态。',
5104     children: [
5105         _buildToggleTile(
5106             icon: Icons.music_note_outlined,
5107             title: '铃声提醒',
5108             subtitle: '有新订单时播放提示音。',
5109             value: notificationSettings.soundEnabled,
5110             onChanged: notificationSettingsNotifier.setSoundEnabled,
5111         ),
```

=====
第 43 页 第 5112~5161 行
=====

```
5112         const SizedBox(height: AppSpacing.md),
5113         _buildToggleTile(
5114             icon: Icons.record_voice_over_outlined,
5115             title: '语音播报',
5116             subtitle: '有新订单时播报订单号和金额。',
5117             value: notificationSettings.voiceEnabled,
5118             onChanged: notificationSettingsNotifier.setVoiceEnabled,
5119         ),
5120         const SizedBox(height: AppSpacing.md),
5121         _buildToggleTile(
5122             icon: Icons.flash_auto_outlined,
5123             title: '自动接单',
5124             subtitle: '新订单到达后自动确认接单; 果失败, 订单仍会保留为待处理。',
5125             value: notificationSettings.autoAcceptEnabled,
5126             onChanged: notificationSettingsNotifier.setAutoAcceptEnabled,
5127         ),
5128         const SizedBox(height: AppSpacing.md),
```

```
5129         _buildToggleTile(  
5130             icon: Icons.print_outlined,  
5131             title: '接单后自动打印',  
5132             subtitle: '手工接单或自动接单成功后, 已连接蓝牙打印机则自动打印小票。',  
5133             value: notificationSettings.autoPrintAfterAcceptEnabled,  
5134             onChanged: notificationSettingsNotifier.setAutoPrintAfterAcceptEnabled,  
5135         ),  
5136         const SizedBox(height: AppSpacing.md),  
5137         _buildActionTile(  
5138             context: context,  
5139             icon: Icons.shutter_speed_outlined,  
5140             title: '自启动与保活设置',  
5141             subtitle: '果后台偶尔漏提醒, 请按品牌指引完成系统设置。',  
5142             badge: const MerchantStatusBadge(  
5143                 label: '需检查',  
5144                 tone: MerchantStatusTone.warning,  
5145             ),  
5146             onTap: () => context.push('/settings/permission-guide'),  
5147         ),  
5148     ],  
5149 ),  
5150 const SizedBox(height: AppSpacing.lg),  
5151 _buildSectionCard(  
5152     title: '账号与应用',  
5153     description: isAuthenticated  
5154         ? '查看版本、帮助信息, 或退出当前商户账号。'  
5155         : '查看版本、帮助信息, 或先完成商户绑定后开始接单。',  
5156     children: [  
5157         _buildActionTile(  
5158             context: context,  
5159             icon: Icons.info_outline,  
5160             title: '关于乐客来福',  
5161             subtitle: '查看版本信息、检查更新和协议入口。',
```

=====
第 44 页 第 5162~5211 行
=====

```
5162             onTap: () => context.push('/settings/about'),  
5163         ),  
5164         const SizedBox(height: AppSpacing.md),  
5165         if (isAuthenticated)  
5166             _buildActionTile(  
5167                 context: context,  
5168                 icon: Icons.logout,  
5169                 iconColor: AppColors.tertiary,  
5170                 title: '退出登录',  
5171                 subtitle: '退出后需要重新使用绑定码登录。',  
5172                 titleColor: AppColors.tertiary,  
5173                 onTap: () async {
```



```
5174         final confirm = await showDialog<bool>(  
5175             context: context,  
5176             builder: (context) => AlertDialog(  
5177                 title: const Text('退出登录'),  
5178                 content: const Text('确定要退出当前商户账号吗? '),  
5179                 actions: [  
5180                     TextButton(  
5181                         onPressed: () => Navigator.pop(context, false),  
5182                         child: const Text('取消'),  
5183                     ),  
5184                     TextButton(  
5185                         onPressed: () => Navigator.pop(context, true),  
5186                         child: const Text('确定退出'),  
5187                     ),  
5188                 ],  
5189             ),  
5190         );  
5191         if (confirm == true) {  
5192             ref.read(workingStatusProvider.notifier).setStatus(false);  
5193             ref.read(orderProvider.notifier).clearOrders();  
5194             ref.read(authProvider.notifier).logout();  
5195         }  
5196     },  
5197 )  
5198 else  
5199     _buildActionTile(  
5200         context: context,  
5201         icon: Icons.login,  
5202         title: '绑定商户',  
5203         subtitle: '输入微信小程序生成的 6 位绑定码后开始接单。',  
5204         onTap: () => context.push('/login'),  
5205     ),  
5206 ],  
5207 ),  
5208 const SizedBox(height: AppSpacing.xl),  
5209 ],  
5210 ),  
5211 ),
```

=====
第 45 页 第 5212~5261 行
=====

```
5212     ],  
5213 ),  
5214 );  
5215 }  
5216  
5217 Widget _buildToggleTile({  
5218     required IconData icon,
```

```
5219     required String title,
5220     required String subtitle,
5221     required bool value,
5222     required ValueChanged<bool> onChanged,
5223   }) {
5224     return Container(
5225       padding: const EdgeInsets.all(AppSpacing.lg),
5226       decoration: BoxDecoration(
5227         color: AppColors.surfaceLow,
5228         borderRadius: BorderRadius.circular(AppRadius.lg),
5229       ),
5230       child: Row(
5231         crossAxisAlignment: CrossAxisAlignment.start,
5232         children: [
5233           Icon(icon, color: AppColors.primary),
5234           const SizedBox(width: AppSpacing.md),
5235           Expanded(
5236             child: Column(
5237               crossAxisAlignment: CrossAxisAlignment.start,
5238               children: [
5239                 Text(
5240                   title,
5241                   style: const TextStyle(
5242                     fontSize: 15,
5243                     fontWeight: FontWeight.w700,
5244                   ),
5245                 ),
5246                 const SizedBox(height: AppSpacing.xs),
5247                 Text(
5248                   subtitle,
5249                   style: const TextStyle(
5250                     color: AppColors.onSurfaceVariant,
5251                     height: 1.45,
5252                   ),
5253                 ),
5254               ],
5255             ),
5256           ),
5257           const SizedBox(width: AppSpacing.md),
5258           Switch(
5259             value: value,
5260             onChanged: onChanged,
5261             activeThumbColor: AppColors.surfaceLowest,
```

第 46 页 第 5262~5311 行

```
5262           activeTrackColor: AppColors.primaryContainer,
5263         ),
```

```
5264     ],
5265   ),
5266 );
5267 }
5268
5269 Widget _buildSectionCard({
5270   required String title,
5271   required String description,
5272   required List<Widget> children,
5273 }) {
5274   return Card(
5275     child: Padding(
5276       padding: const EdgeInsets.all(AppSpacing.xl),
5277       child: Column(
5278         crossAxisAlignment: CrossAxisAlignment.start,
5279         children: [
5280           Text(
5281             title,
5282             style: const TextStyle(
5283               fontSize: 18,
5284               fontWeight: FontWeight.w700,
5285             ),
5286           ),
5287           const SizedBox(height: AppSpacing.xs),
5288           Text(
5289             description,
5290             style: const TextStyle(
5291               color: AppColors.onSurfaceVariant,
5292               height: 1.45,
5293             ),
5294           ),
5295           const SizedBox(height: AppSpacing.lg),
5296           ...children,
5297         ],
5298       ),
5299     ),
5300 );
5301 }
5302
5303 Widget _buildActionTile({
5304   required BuildContext context,
5305   required IconData icon,
5306   required String title,
5307   required String subtitle,
5308   VoidCallback? onTap,
5309   Widget? badge,
5310   Color? iconColor,
5311   Color? titleColor,
```

第 47 页 第 5312~5361 行

```
5312    }) {
5313        final content = Container(
5314            padding: const EdgeInsets.all(AppSpacing.lg),
5315            decoration: BoxDecoration(
5316                color: AppColors.surfaceLow,
5317                borderRadius: BorderRadius.circular(AppRadius.lg),
5318            ),
5319            child: Row(
5320                crossAxisAlignment: CrossAxisAlignment.start,
5321                children: [
5322                    Icon(icon, color: iconColor ?? AppColors.primary),
5323                    const SizedBox(width: AppSpacing.md),
5324                    Expanded(
5325                        child: Column(
5326                            crossAxisAlignment: CrossAxisAlignment.start,
5327                            children: [
5328                                Text(
5329                                    title,
5330                                    style: TextStyle(
5331                                        fontSize: 15,
5332                                        fontWeight: FontWeight.w700,
5333                                        color: titleColor ?? AppColors.onSurface,
5334                                    ),
5335                                ),
5336                                const SizedBox(height: AppSpacing.xs),
5337                                Text(
5338                                    subtitle,
5339                                    style: const TextStyle(
5340                                        color: AppColors.onSurfaceVariant,
5341                                        height: 1.45,
5342                                    ),
5343                                ),
5344                            ],
5345                        ),
5346                    ),
5347                    const SizedBox(width: AppSpacing.md),
5348                    badge ??
5349                    const Icon(
5350                        Icons.chevron_right,
5351                        color: AppColors.onSurfaceVariant,
5352                    ),
5353                ],
5354            ),
5355        );
5356
5357        if (onTap == null) {
```

```
5358         return content;
5359     }
5360
5361     return InkWell(
```

第 48 页 第 5362~5411 行

```
5362         onTap: onTap,
5363         borderRadius: BorderRadius.circular(AppRadius.lg),
5364         child: content,
5365     );
5366 }
5367 }
5368
5369
5370 // =====
5371 // 文件: lib/features/update/update_dialog.dart
5372 // =====
5373 import 'package:flutter/material.dart';
5374 import 'package:merchant_app/config/theme.dart';
5375 import 'package:url_launcher/url_launcher.dart';
5376
5377 class UpdateDialog extends StatelessWidget {
5378     final String versionName;
5379     final String changelog;
5380     final String downloadUrl;
5381     final bool isForce;
5382
5383     const UpdateDialog({
5384         super.key,
5385         required this.versionName,
5386         required this.changelog,
5387         required this.downloadUrl,
5388         required this.isForce,
5389     });
5390
5391     Future<void> _launchUrl(BuildContext context) async {
5392         if (downloadUrl.trim().isEmpty) {
5393             ScaffoldMessenger.of(context).showSnackBar(
5394                 const SnackBar(content: Text('当前未提供下载地址')),
5395             );
5396             return;
5397         }
5398         final url = Uri.parse(downloadUrl);
5399         if (await canLaunchUrl(url)) {
5400             await launchUrl(url, mode: LaunchMode.externalApplication);
5401             return;
5402         }
```

```
5403
5404     if (context.mounted) {
5405         ScaffoldMessenger.of(context).showSnackBar(
5406             const SnackBar(content: Text('无法打开下载地址')),
5407         );
5408     }
5409 }
5410
5411 @override
```

第 49 页 第 5412~5461 行

```
5412 Widget build(BuildContext context) {
5413     return PopScope(
5414         canPop: !isForce,
5415         child: AlertDialog(
5416             title: Text('发现新版本 V$versionName'),
5417             content: Column(
5418                 mainAxisAlignment: MainAxisAlignment.min,
5419                 crossAxisAlignment: CrossAxisAlignment.start,
5420                 children: [
5421                     Container(
5422                         width: double.infinity,
5423                         padding: const EdgeInsets.all(AppSpacing.lg),
5424                         decoration: BoxDecoration(
5425                             color: AppColors.surfaceLow,
5426                             borderRadius: BorderRadius.circular(AppRadius.lg),
5427                         ),
5428                     child: Column(
5429                         crossAxisAlignment: CrossAxisAlignment.start,
5430                         children: [
5431                             const Text(
5432                                 '新版内容',
5433                                 style: TextStyle(fontWeight: FontWeight.w700),
5434                             ),
5435                             const SizedBox(height: AppSpacing.sm),
5436                             Text(
5437                                 changelog,
5438                                 style: const TextStyle(height: 1.5),
5439                             ),
5440                         ],
5441                     ),
5442                 ],
5443             ),
5444             actions: [
5445                 if (!isForce)
5446                     TextButton(
```

```
5448         onPressed: () => Navigator.pop(context),
5449         child: const Text('稍后提醒', style: TextStyle(color: Colors.grey)),
5450     ),
5451     ElevatedButton(
5452       onPressed: () {
5453         _launchUrl(context);
5454         if (!isForce) {
5455           Navigator.pop(context);
5456         }
5457       },
5458       child: const Text('前往下载'),
5459     ),
5460   ],
5461 ),
```

第 50 页 第 5462~5511 行

```
5462   );
5463 }
5464 }
5465
5466
5467
5468 // =====
5469 // 文件: lib/features/update/update_service.dart
5470 // =====
5471 import 'package:flutter/material.dart';
5472 import 'package:flutter_riverpod/flutter_riverpod.dart';
5473 import 'package:package_info_plus/package_info_plus.dart';
5474 import 'package:merchant_app/core/network/api_provider.dart';
5475 import 'package:merchant_app/core/utils/error_handler.dart';
5476 import 'update_dialog.dart';
5477
5478 final updateServiceProvider = Provider((ref) {
5479   final apiClient = ref.watch(apiClientProvider);
5480   return UpdateService(apiClient);
5481 });
5482
5483 class UpdateService {
5484   UpdateService(this._apiClient);
5485
5486   final dynamic _apiClient;
5487
5488   Future<void> checkForUpdates(BuildContext context, {bool showToastIfLatest = false}) async {
5489     try {
5490       final packageInfo = await PackageInfo.fromPlatform();
5491       final currentCode = int.tryParse(packageInfo.buildNumber) ?? 1;
5492
```

```
5493     final response = await _apiClient.get(
5494         '/app/version/latest',
5495         queryParameters: {
5496             'platform': 'android',
5497             'channel': 'merchant_app',
5498             'package_name': 'com.merrydance.locallife.merchant',
5499             'version_code': currentCode,
5500             'version_name': packageInfo.version,
5501         },
5502     );
5503
5504     final responseData = response.data;
5505     final data = responseData is Map<String, dynamic>
5506         ? (responseData['data'] as Map<String, dynamic>?) ?? responseData
5507         : <String, dynamic>{};
5508
5509     final serverCode = (data['version_code'] as num?)?.toInt() ?? currentCode;
5510
5511     if (serverCode > currentCode) {
```

=====

第 51 页 第 5512~5561 行

=====

```
5512         if (!context.mounted) return;
5513
5514         showDialog(
5515             context: context,
5516             barrierDismissible: !(data['is_force'] as bool? ?? false),
5517             builder: (context) => UpdateDialog(
5518                 versionName: data['version_name']?.toString() ?? packageInfo.version,
5519                 changelog: data['changelog']?.toString() ?? '本次版本包含若干稳定性修复。',
5520                 downloadUrl: data['download_url']?.toString() ?? '',
5521                 isForce: data['is_force'] as bool? ?? false,
5522             ),
5523         );
5524     } else {
5525         if (showToastIfLatest && context.mounted) {
5526             ScaffoldMessenger.of(context).showSnackBar(
5527                 const SnackBar(content: Text('已经是最新版本')),
5528             );
5529         }
5530     }
5531 } catch (e) {
5532     if (!context.mounted) {
5533         return;
5534     }
5535
5536     final message = _isUpdateApiMissing(e)
5537         ? '当前版本暂未接入在线升级, 请等待后端接口上线'
```



```
5538         : ErrorHandler.getErrorMessage(e);
5539
5540     if (showToastIfLatest) {
5541         ScaffoldMessenger.of(context).showSnackBar(
5542             SnackBar(content: Text(message)),
5543         );
5544     }
5545 }
5546 }
5547
5548 bool _isUpdateApiMissing(Object error) {
5549     final message = error.toString();
5550     return message.contains('404');
5551 }
5552 }
5553
5554
5555 // =====
5556 // 文件: lib/main.dart
5557 // =====
5558 import 'package:flutter/material.dart';
5559 import 'package:flutter_riverpod/flutter_riverpod.dart';
5560 import 'package:merchant_app/app.dart';
5561 import 'package:merchant_app/core/audio/tts_service.dart';
```

=====

第 52 页 第 5562~5611 行

=====

```
5562 import 'package:merchant_app/core/push/push_provider.dart';
5563 import 'package:merchant_app/core/network/ws_provider.dart';
5564 import 'package:merchant_app/core/service/order_poller.dart';
5565 import 'package:merchant_app/core/service/foreground_service.dart';
5566 import 'package:merchant_app/features/order/order_alert_coordinator.dart';
5567
5568 // Removed global key from here
5569
5570 class MerchantAppBootstrap extends ConsumerWidget {
5571     const MerchantAppBootstrap({super.key});
5572
5573     @override
5574     Widget build(BuildContext context, WidgetRef ref) {
5575         ref.watch(wsConnectionManagerProvider);
5576         ref.watch(orderPollerManagerProvider);
5577         return const MerchantApp();
5578     }
5579 }
5580
5581 void main() async {
5582     WidgetsFlutterBinding.ensureInitialized();
```

```
5583
5584 // Initialize Services
5585 await TtsService.init();
5586 await MerchantForegroundService.init();
5587
5588 final container = ProviderContainer();
5589 await container.read(messageDeduplicatorProvider).ensureInitialized();
5590 final localNotificationService = container.read(localNotificationServiceProvider);
5591 await localNotificationService.init();
5592 await localNotificationService.ensureNotificationPermission();
5593
5594 // Initialize Push Manager
5595 final pushManager = container.read(pushManagerProvider);
5596 await pushManager.init();
5597
5598 // Initialize WebSocket Manager
5599 final wsClient = container.read(wsClientProvider);
5600 final orderAlertCoordinator = container.read(orderAlertCoordinatorProvider);
5601
5602 localNotificationService.onNotificationTap = orderAlertCoordinator.handleNotificationTap;
5603
5604 // Handle new order push
5605 pushManager.onNewOrder = orderAlertCoordinator.handleIncomingOrder;
5606 pushManager.onNotificationOpened = orderAlertCoordinator.handleNotificationTap;
5607
5608 // Handle new order WebSocket
5609 wsClient.onNewOrder = (message) {
5610   orderAlertCoordinator.handleIncomingOrder(
5611     message,
```

=====

第 53 页 第 5612~5661 行

=====

```
5612     showLocalNotification: true,
5613   );
5614 };
5615
5616 runApp(
5617   UncontrolledProviderScope(
5618     container: container,
5619     child: const MerchantAppBootstrap(),
5620   ),
5621 );
5622 }
5623
5624
5625 // =====
5626 // 文件: lib/models/order.dart
5627 // =====
```

```
5628 import 'package:intl/intl.dart';
5629
5630 enum OrderStatus {
5631     pending('待接单'),
5632     accepted('已接单'),
5633     preparing('制作中'),
5634     delivering('配送中'),
5635     completed('已完成'),
5636     cancelled('已取消');
5637
5638     final String label;
5639     const OrderStatus(this.label);
5640
5641     static OrderStatus fromString(String value) {
5642         return OrderStatus.values.firstWhere(
5643             (e) => e.name == value.toLowerCase(),
5644             orElse: () => OrderStatus.pending,
5645         );
5646     }
5647 }
5648
5649 class OrderModel {
5650     final String id;
5651     final String orderNum;
5652     final double amount;
5653     final OrderStatus status;
5654     final DateTime createdAt;
5655     final String? userName;
5656     final String? userPhone;
5657     final List<OrderItem> items;
5658     final String? note;
5659
5660     OrderModel({
5661         required this.id,
```

=====

第 54 页 第 5662~5711 行

=====

```
5662         required this.orderNum,
5663         required this.amount,
5664         required this.status,
5665         required this.createdAt,
5666         this.userName,
5667         this.userPhone,
5668         required this.items,
5669         this.note,
5670     });
5671
5672     factory OrderModel.fromJson(Map<String, dynamic> json) {
```

```
5673     return OrderModel(  
5674         id: json['id']?.toString() ?? '',  
5675         orderNum: json['order_num']?.toString() ?? '',  
5676         amount: (json['amount'] as num?)?.toDouble() ?? 0.0,  
5677         status: OrderStatus.fromString(json['status']?.toString() ?? 'pending'),  
5678         createdAt: DateTime.tryParse(json['created_at']?.toString() ?? '') ?? DateTime.now(),  
5679         userName: json['user_name'],  
5680         userPhone: json['user_phone'],  
5681         items: (json['items'] as List?)?.map((i) => OrderItem.fromJson(i)).toList() ?? [],  
5682         note: json['note'],  
5683     );  
5684 }  
5685  
5686 String get formattedDate => DateFormat('MM-dd HH:mm').format(createdAt);  
5687 }  
5688  
5689 class OrderItem {  
5690     final String name;  
5691     final int quantity;  
5692     final double price;  
5693  
5694     OrderItem({  
5695         required this.name,  
5696         required this.quantity,  
5697         required this.price,  
5698     });  
5699  
5700     factory OrderItem.fromJson(Map<String, dynamic> json) {  
5701         return OrderItem(  
5702             name: json['name']?.toString() ?? '',  
5703             quantity: json['quantity'] as int? ?? 1,  
5704             price: (json['price'] as num?)?.toDouble() ?? 0.0,  
5705         );  
5706     }  
5707 }  
5708  
5709  
5710 // =====  
5711 // 文件: lib/models/push_message.dart
```

=====

第 55 页 第 5712~5761 行

=====

```
5712 // =====  
5713 class PushMessage {  
5714     final String messageId;  
5715     final String orderId;  
5716     final String orderNumber;  
5717     final String title;
```

```
5718     final String content;
5719     final double amount;
5720     final String shopName;
5721     final DateTime timestamp;
5722
5723     PushMessage({
5724         required this.messageId,
5725         required this.orderId,
5726         this.orderNumber = '',
5727         required this.title,
5728         required this.content,
5729         required this.amount,
5730         required this.shopName,
5731         DateTime? timestamp,
5732     }) : timestamp = timestamp ?? DateTime.now();
5733
5734     factory PushMessage.fromJson(Map<String, dynamic> json) {
5735         return PushMessage(
5736             messageId: json['message_id'] ?? '',
5737             orderId: json['order_id'] ?? '',
5738             orderNumber: json['order_num'] ?? json['order_number'] ?? json['order_id'] ?? '',
5739             title: json['title'] ?? '',
5740             content: json['content'] ?? '',
5741             amount: double.tryParse(json['amount']?.toString() ?? '0') ?? 0.0,
5742             shopName: json['shop_name'] ?? '',
5743         );
5744     }
5745
5746     factory PushMessage.fromOrder(
5747         dynamic order, {
5748             required String shopName,
5749         }) {
5750         return PushMessage(
5751             messageId: 'polled-${order.id}',
5752             orderId: order.id,
5753             orderNumber: order.orderNum,
5754             title: '您有新的订单',
5755             content: '订单号 ${order.orderNum.isNotEmpty ? order.orderNum : order.id}, 请及时处理',
5756             amount: order.amount,
5757             shopName: shopName,
5758             timestamp: order.createdAt,
5759         );
5760     }
5761
```

第 56 页 第 5762~5811 行

```
5762     PushMessage withOrderNumber(String nextOrderNumber) {
```

```
5763     return PushMessage(  
5764         messageId: messageId,  
5765         orderId: orderId,  
5766         orderNumber: nextOrderNumber,  
5767         title: title,  
5768         content: content,  
5769         amount: amount,  
5770         shopName: shopName,  
5771         timestamp: timestamp,  
5772     );  
5773 }  
5774  
5775 Map<String, dynamic> toJson() {  
5776     return {  
5777         'message_id': messageId,  
5778         'order_id': orderId,  
5779         'order_num': orderNumber,  
5780         'title': title,  
5781         'content': content,  
5782         'amount': amount,  
5783         'shop_name': shopName,  
5784     };  
5785 }  
5786  
5787 String get displayOrderNumber => orderNumber.isNotEmpty ? orderNumber : orderId;  
5788  
5789 int get notificationId => Object.hash(orderId, messageId);  
5790 }  
5791  
5792  
5793 // =====  
5794 // 文件: lib/widgets/merchant_content_shell.dart  
5795 // =====  
5796 import 'package:flutter/material.dart';  
5797 import 'package:merchant_app/config/theme.dart';  
5798  
5799 class MerchantContentShell extends StatelessWidget {  
5800     const MerchantContentShell({  
5801         required this.child,  
5802         super.key,  
5803         this.maxWidth = 420,  
5804         this.padding = const EdgeInsets.symmetric(  
5805             horizontal: AppSpacing.xl,  
5806             vertical: AppSpacing.xl,  
5807         ),  
5808     });  
5809  
5810     final Widget child;  
5811     final double maxWidth;
```

第 57 页 第 5812~5861 行

```
5812   final EdgeInsetsGeometry padding;
5813
5814   @override
5815   Widget build(BuildContext context) {
5816     return SafeArea(
5817       child: Align(
5818         alignment: Alignment.topCenter,
5819         child: ConstrainedBox(
5820           constraints: BoxConstraints(maxWidth: maxWidth),
5821           child: Padding(
5822             padding: padding,
5823             child: child,
5824           ),
5825         ),
5826       ),
5827     );
5828   }
5829 }
5830
5831 // =====
5832 // 文件: lib/widgets/merchant_primary_button.dart
5833 // =====
5834 import 'package:flutter/material.dart';
5835
5836 class MerchantPrimaryButton extends StatelessWidget {
5837   const MerchantPrimaryButton({
5838     required this.label,
5839     required this.onPressed,
5840     super.key,
5841     this.expand = false,
5842     this.maxWidth = 360,
5843     this.icon,
5844     this.isLoading = false,
5845   });
5846
5847   final String label;
5848   final VoidCallback? onPressed;
5849   final bool expand;
5850   final double maxWidth;
5851   final Widget? icon;
5852   final bool isLoading;
5853
5854   @override
5855   Widget build(BuildContext context) {
5856     final labelWidget = isLoading
```

```
5857      ? const SizedBox(  
5858          width: 18,  
5859          height: 18,  
5860          child: CircularProgressIndicator(strokeWidth: 2),  
5861      )
```

第 58 页 第 5862~5911 行

```
5862      : Text(label);  
5863  
5864      final button = icon == null || isLoading  
5865      ? ElevatedButton(  
5866          onPressed: isLoading ? null : onPressed,  
5867          child: labelWidget,  
5868      )  
5869      : ElevatedButton.icon(  
5870          onPressed: onPressed,  
5871          icon: icon!,  
5872          label: labelWidget,  
5873      );  
5874  
5875      if (expand) {  
5876          return SizedBox(width: double.infinity, child: button);  
5877      }  
5878  
5879      return Align(  
5880          alignment: Alignment.center,  
5881          child: ConstrainedBox(  
5882              constraints: BoxConstraints(maxWidth: maxWidth),  
5883              child: SizedBox(width: double.infinity, child: button),  
5884          ),  
5885      );  
5886  }  
5887 }  
5888  
5889 // =====  
5890 // 文件: lib/widgets/merchant_secondary_button.dart  
5891 // =====  
5892 import 'package:flutter/material.dart';  
5893 import 'package:merchant_app/config/theme.dart';  
5894  
5895 class MerchantSecondaryButton extends StatelessWidget {  
5896     const MerchantSecondaryButton({  
5897         required this.label,  
5898         required this.onPressed,  
5899         super.key,  
5900         this.expand = false,  
5901         this.icon,
```



```
5902     });
5903
5904     final String label;
5905     final VoidCallback? onPressed;
5906     final bool expand;
5907     final Widget? icon;
5908
5909     @override
5910     Widget build(BuildContext context) {
5911         final button = icon == null
```

第 59 页 第 5912~5961 行

```
5912         ? OutlinedButton(
5913             onPressed: onPressed,
5914             child: Text(label),
5915         )
5916         : OutlinedButton.icon(
5917             onPressed: onPressed,
5918             icon: icon!,
5919             label: Text(label),
5920         );
5921
5922         final themed = Theme(
5923             data: Theme.of(context).copyWith(
5924                 outlinedButtonTheme: OutlinedButtonThemeData(
5925                     style: OutlinedButton.styleFrom(
5926                         minimumSize: const Size(0, 56),
5927                         padding: const EdgeInsets.symmetric(
5928                             horizontal: AppSpacing.xl,
5929                             vertical: AppSpacing.lg,
5930                         ),
5931                         foregroundColor: AppColors.onSurface,
5932                         side: const BorderSide(color: AppColors.outlineVariant),
5933                         shape: RoundedRectangleBorder(
5934                             borderRadius: BorderRadius.circular(AppRadius.lg),
5935                         ),
5936                     ),
5937                 ),
5938             ),
5939             child: button,
5940         );
5941
5942         if (expand) {
5943             return SizedBox(width: double.infinity, child: themed);
5944         }
5945
5946         return Align(
```

```
5947     alignment: Alignment.center,
5948     child: SizedBox(width: double.infinity, child: themed),
5949   );
5950 }
5951 }
5952
5953 // =====
5954 // 文件: lib/widgets/merchant_status_badge.dart
5955 // =====
5956 import 'package:flutter/material.dart';
5957 import 'package:merchant_app/config/theme.dart';
5958
5959 enum MerchantStatusTone { neutral, positive, warning, danger }
5960
5961 class MerchantStatusBadge extends StatelessWidget {
```

第 60 页 第 5962~6011 行

```
5962   const MerchantStatusBadge({
5963     required this.label,
5964     required this.tone,
5965     super.key,
5966   });
5967
5968   final String label;
5969   final MerchantStatusTone tone;
5970
5971   @override
5972   Widget build(BuildContext context) {
5973     final palette = switch (tone) {
5974       MerchantStatusTone.positive => (
5975         background: const Color(0xFFE5F5EA),
5976         foreground: AppColors.primary,
5977       ),
5978       MerchantStatusTone.warning => (
5979         background: AppColors.warningSoft,
5980         foreground: AppColors.secondary,
5981       ),
5982       MerchantStatusTone.danger => (
5983         background: AppColors.dangerSoft,
5984         foreground: AppColors.tertiary,
5985       ),
5986       MerchantStatusTone.neutral => (
5987         background: AppColors.surfaceLow,
5988         foreground: AppColors.onSurfaceVariant,
5989       ),
5990     };
5991
```

```
5992     return Container(  
5993         padding: const EdgeInsets.symmetric(  
5994             horizontal: AppSpacing.md,  
5995             vertical: AppSpacing.sm,  
5996         ),  
5997         decoration: BoxDecoration(  
5998             color: palette.background,  
5999             borderRadius: BorderRadius.circular(AppRadius.pill),  
6000         ),  
6001         child: Text(  
6002             label,  
6003             style: TextStyle(  
6004                 color: palette.foreground,  
6005                 fontSize: 12,  
6006                 fontWeight: FontWeight.w700,  
6007             ),  
6008         ),  
6009     );  
6010 }  
6011 }
```